Weather Information App

A Java Swing application with live weather data from the Open-Meteo API

Doston Abduraimov | University of the People | October 2026

1. Overview

This report accompanies the Weather Information App assignment. The app shows real-time weather for any city or pair of coordinates: the current conditions, a 5-day forecast, weather icons, unit switches, a history of searches and a background that follows the time of day at the searched place. It is written in **Java Swing**, uses the free **Open-Meteo** weather API (no API key and no external libraries are needed), and consists of 30 classes (2,656 lines with comments). The report contains the README, screenshots of the running application, the test results and the complete source code.

How each requirement is met

Requirement	Where / how it is met	See
API integration	`WeatherService` calls the Open-Meteo geocoding and forecast APIs over HTTPS with `java.net.http.HttpClient`; `Json` reads the replies.	Fig. 2, Sec. 2
GUI (Swing), location input	`WeatherFrame` with a search field (city or coordinates, validated by `InputParser`), a Search button, unit switches and a message bar.	Fig. 1-4
Weather information	`CurrentWeatherPanel`: temperature, feels like, humidity, wind speed and direction, conditions, pressure, sunrise and sunset, local time.	Fig. 2
Icons	`WeatherIcon` draws eight weather pictures (sun or moon variants) with Java 2D; `WeatherCode` maps API codes to pictures.	Fig. 2-4
Forecast	`ForecastPanel`: five day cards with icon, high / low, conditions, precipitation chance.	Fig. 2-3
Unit conversion	`TemperatureUnit` (°C, °F) and `WindSpeedUnit` (km/h, mph, m/s, knots); `SegmentedControl` switches; no new API call.	Fig. 3
Error handling	`WeatherException` with four types; `InputParser` validation; timeouts, HTTP errors and bad replies handled in `WeatherService`; `StatusBar` shows them.	Fig. 6
History with timestamps	`SearchHistory` (saved to a file) and `HistoryPanel`; click to search again.	Fig. 2-4
Dynamic background	`TimeOfDay` decides sunrise, day, sunset or night from the place's sunrise and sunset; `BackgroundPanel` paints the sky, sun, moon and stars.	Fig. 5
Commented code, README	Javadoc on every class and method that is not obvious; README in Section 2.	Sec. 2, 5

2. README

A desktop weather app written in Java. Type a city (or coordinates) and it shows the current weather, a 5-day forecast, weather icons and a background that follows the time of day at that place. It is built with **Java Swing** and gets its data from the free **Open-Meteo** weather API. It needs no libraries and no API key.

Features

Requirement	How the app meets it
API integration	`WeatherService` calls the Open-Meteo **geocoding** API (name to coordinates) and **forecast** API (current weather and daily forecast) over HTTPS and reads the JSON replies.
GUI design (Swing)	One window with a search box, a **Search** button, unit switches, a message bar, the weather cards and a history list. Enter also starts a search.
Location input	A city name (`Paris`, `Paris, France`, `Paris, Texas`) **or** coordinates (`48.85, 2.35`).
Weather information	Temperature, feels-like temperature, humidity, wind speed and direction, conditions, pressure, sunrise and sunset, and the local time of the searched place.
Icons	Sun, moon, partly cloudy, clouds, fog, drizzle, rain, snow and thunderstorm icons, drawn with Java 2D (sun by day, moon by night).
Forecast	Five day cards: weekday and date, icon, high / low temperature, conditions and chance of precipitation.
Unit conversion	Temperature: **°C / °F**. Wind speed: **km/h, mph, m/s, knots**. Switching is instant and does not call the API again.
Error handling	Invalid input (empty, unusable characters, bad coordinates), unknown places, no internet connection, timeouts, and API errors each show a clear message in a red bar. The app never crashes and keeps the last result on screen.
History	The last 20 searches with date and time, saved to a file so they survive a restart. Click an entry to look it up again; **Clear history** empties the list.
Dynamic background	The sky changes between **sunrise**, **day**, **sunset** and **night**, based on the sunrise and sunset times of the searched place (sun, moon and stars are drawn).

Requirements

- **Java 17 or newer** (developed and tested with OpenJDK 21). Swing is part of the JDK.
- An internet connection (the weather data comes from the web).
- No API key, no external libraries and no build tool.

How to run

Open a terminal in the project folder (the one that contains `src/`).

1. Compile

```
javac -encoding UTF-8 -d out src/weatherapp/*.java
```

2. Start the app

```
java -cp out weatherapp.WeatherApp
```

How to use it

1. Type a place in the search box: a **city** such as `Tashkent`, a city with a country or state to be exact such as `Paris, France` or `Paris, Texas`, or **coordinates** as `latitude, longitude` such as `48.85, 2.35` (south and west are negative: `-33.87, 151.21`).
2. Press **Search** (or Enter). The message bar shows the progress and then the result.

3. Read the **current weather** card and the **5-day forecast**. Hover over a forecast icon for the conditions.
4. Use the **Temperature** and **Wind speed** switches to change units. The display updates at once.
5. Click an entry under **Recent searches** to see that place again. **Clear history** removes all entries.
6. The window background shows what time it is *at the searched place*: a glowing dawn sky, a blue day sky, a red and orange sunset, or a starry night with the moon.

Search history is stored in `.weather-information-app/history.tsv` in your home folder. Start the app with `-Dweatherapp.history=C:\path\to\file.tsv` to use another file.

The weather API and its usage guidelines

The app uses Open-Meteo:

Purpose	Request
Place name to coordinates	`GET https://geocoding-api.open-meteo.com/v1/search?name=<city>&count=1&language=en&format=json`
Weather for coordinates	`GET https://api.open-meteo.com/v1/forecast?latitude=..&longitude=..&current=temperature_2m,relative_humidity_2m,apparent_temperature,is_day,weather_code,surface_pressure,wind_speed_10m,wind_direction_10m&daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max,sunrise,sunset&timezone=auto&forecast_days=5`

API key. Open-Meteo's free API does not use keys, so there is no secret in the code or in the repository, and the project runs as soon as it is compiled. (Services such as OpenWeatherMap need a personal key. `WeatherService` is the only class that knows about the web service, so another provider would only change that class.)

Usage guidelines followed (from open-meteo.com/en/terms and /en/licence, checked on 27 October 2026):

- *Non-commercial use only.* This is a free educational project with no ads or subscriptions.
- *Rate limits* of the free tier are 600 calls per minute, 5,000 per hour and 10,000 per day. The app makes one call per search for coordinates and two for a city name, never polls or refreshes by itself, and ignores a second search while one is still running. Changing units never calls the API.
- *Attribution (CC BY 4.0).* The window always shows a clickable "Weather data by Open-Meteo.com" link next to the data.
- Requests send an identifying `User-Agent` header and use HTTPS.

How it works

Class	Responsibility
`WeatherApp`	Entry point; starts the window on the Swing event thread.
`WeatherFrame`	The main window: controls, searching on a background thread (`SwingWorker`), showing results and errors.
`WeatherService`	Calls the two Open-Meteo APIs and converts the replies into Java objects; turns every failure into a `WeatherException`.
`Json`	A small JSON reader (the project needs no library).
`InputParser`, `LocationQuery`	Validate what the user typed and decide whether it is a city name or coordinates.
`Location`, `CurrentConditions`,	Immutable data records.

`DailyForecast`, `WeatherReport`	
`TemperatureUnit`, `WindSpeedUnit`	Unit conversion. Data is stored in °C and km/h and converted when displayed.
`DisplayFormat`	Turns numbers and dates into display text (English format on every computer).
`WeatherCode`, `IconType`, `WeatherIcon`	Translate weather codes to text and icons; draw the icons with Java 2D.
`TimeOfDay`, `BackgroundPanel`	Choose sunrise, day, sunset or night from the sun times, and paint the matching sky.
`SearchHistory`	The list of recent searches with timestamps, saved to a text file.
`CurrentWeatherPanel`, `ForecastPanel`, `HistoryPanel`, `StatusBar`	The cards of the window.
`RoundedPanel`, `HintTextField`, `AccentButton`, `SegmentedControl`, `ThinScrollBarUI`, `Ui`	Small reusable pieces of the look and feel.

Design decisions:

- **The window never freezes.** Network calls run in a `SwingWorker`; the result is shown back on the event thread. The Search button is disabled while a search runs.
- **Convert at display time.** Storing metric values and converting when drawing makes unit switches instant and avoids repeated rounding errors.
- **One exception type for expected problems.** `WeatherException` carries a type (invalid input, not found, network, service) and a message written for the user. Unexpected bugs are logged to the console and shown as a generic message.
- **Dynamic background by the sun, not the clock.** `TimeOfDay.at(now, sunrise, sunset)` uses the place's own local time and its sunrise and sunset: sunrise is 45 minutes before to 60 minutes after sunrise, sunset is 75 minutes before to 45 minutes after sunset, night is the rest. If the sun times are missing, the clock is used (before the first search the computer's clock is used).

Error messages

Situation	Message (shortened)
Empty search	Invalid input: Please enter a city name or coordinates first.
Unusable characters, e.g. `!!!`	Invalid input: Use only letters, numbers, spaces and hyphens in a city name, or coordinates like 48.85, 2.35.
Coordinates out of range, e.g. `95, 10`	Invalid input: Latitude must be between -90 and 90 degrees.
Unknown place	Location not found: No place called "..." was found. Check the spelling, or add the country.
No internet / server unreachable	Connection problem: Could not reach the weather service. Check your internet connection.
Slow server	Connection problem: The weather service took too long to answer.
HTTP error, rate limit, bad reply	Weather service problem: ...

Tests

```
javac -encoding UTF-8 -d out src/weatherapp/*.java tests/weatherapp/*.java
java -cp out weatherapp.TestRunner
```

`TestRunner` needs no test library. It runs **240 checks**: unit conversion, number and date formatting, weather codes and icons, the time-of-day rules, the JSON reader (including malformed input), input validation, the search history file, the service against a local mock server (every error case: HTTP 400, 429, 500, bad JSON, missing data, timeouts, refused connections, unknown hosts), the window itself (searches, errors, unit switching, backgrounds, history) and the real Open-Meteo API. The results are saved in `tests/output/test_results.txt`.

`java -cp out weatherapp.Screenshots docs/screenshots` starts the real window, drives it and saves the screenshots used in the report (needs a display and an internet connection).

Project layout

```
src/weatherapp/     the application (24 classes)
tests/weatherapp/   TestRunner (automated checks) and Screenshots (screenshot tool)
tests/output/       saved test results
docs/screenshots/   screenshots of the running application
docs/build_report.py   builds the Word/PDF report
```

Credits

Weather data by Open-Meteo.com (CC BY 4.0). Place names come from GeoNames through the Open-Meteo geocoding API.

3. Screenshots of the application

All screenshots are real captures of the running application, taken by `tests/weatherapp/Screenshots.java`, which opens the window, types into it, presses the buttons and copies the pixels from the screen. The weather data is live: it is the weather at the moment the screenshots were taken (27 October 2026).

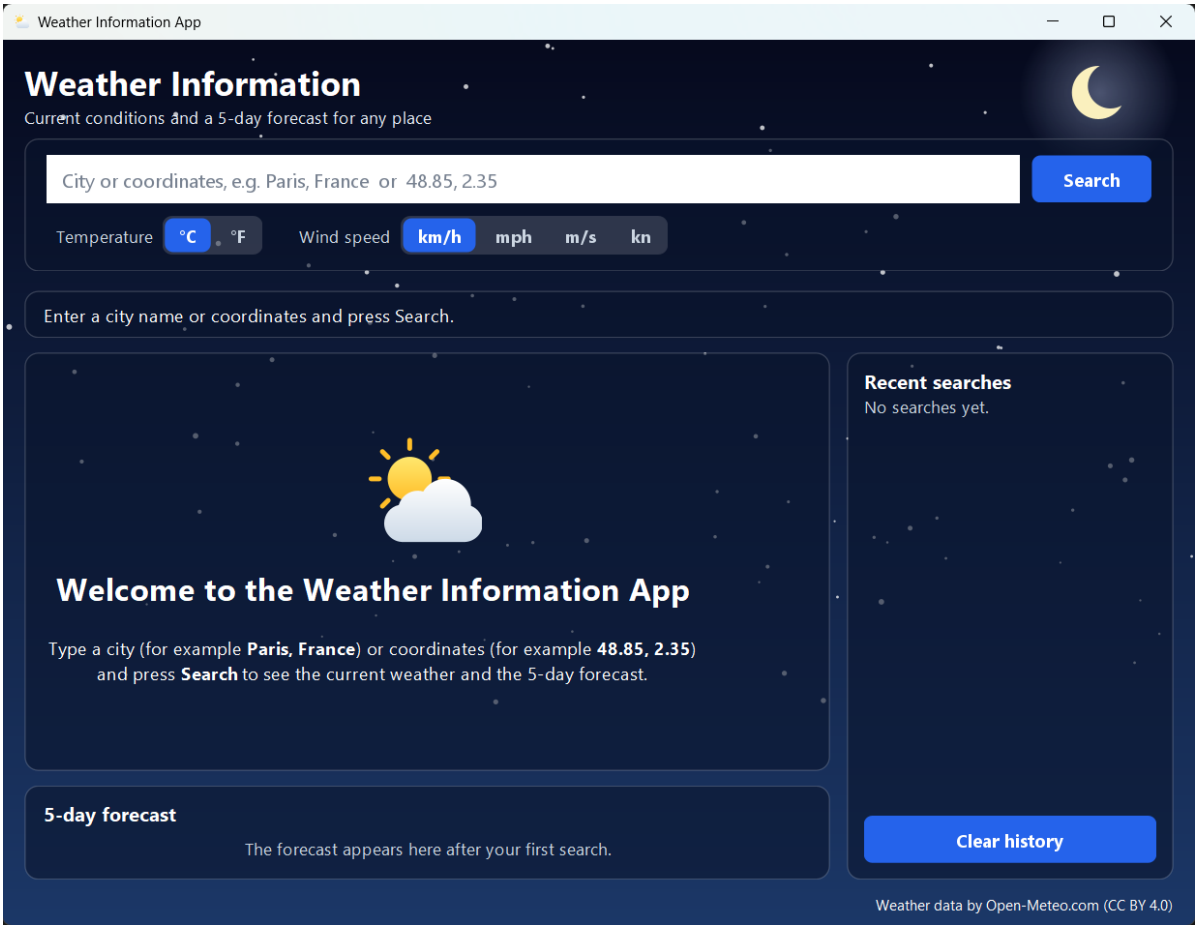


Figure 1. The window at start-up: search box, unit switches, message bar, welcome card and an empty history. The background follows the computer's clock (here: night).



Figure 2. Result for "Los Angeles" (local time 10:00, America/Los_Angeles): current temperature, conditions, feels like, humidity, wind speed and direction, pressure, sunrise and sunset, the 5-day forecast with icons, and the list of recent searches with timestamps.

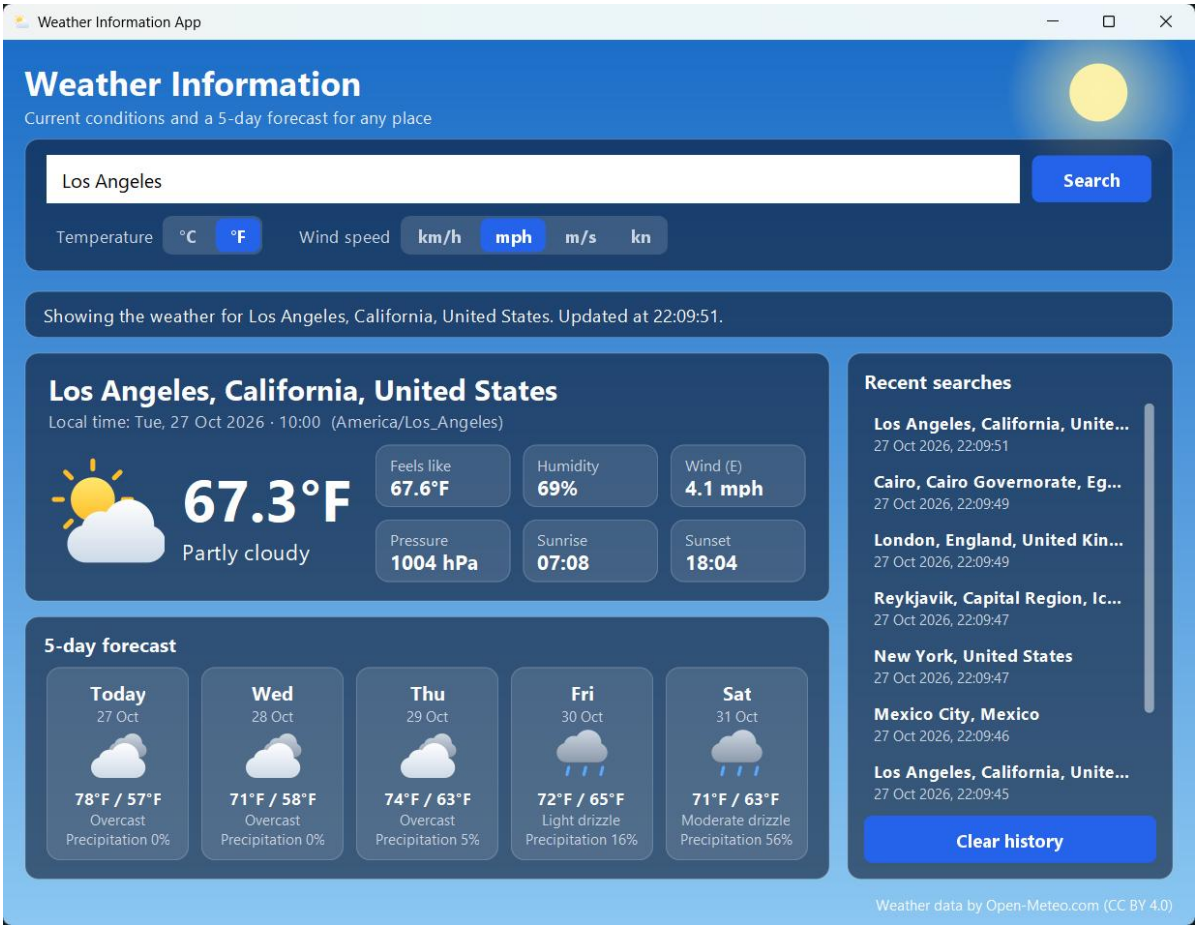


Figure 3. The same result after switching to °F and mph: the current weather and the whole forecast are converted instantly without a new request to the API.



Figure 4. A search by coordinates ("48.8566, 2.3522"). The place is named after the coordinates; the time zone (Europe/Paris) and local time (18:00) come from the API. Wind is shown in knots here.

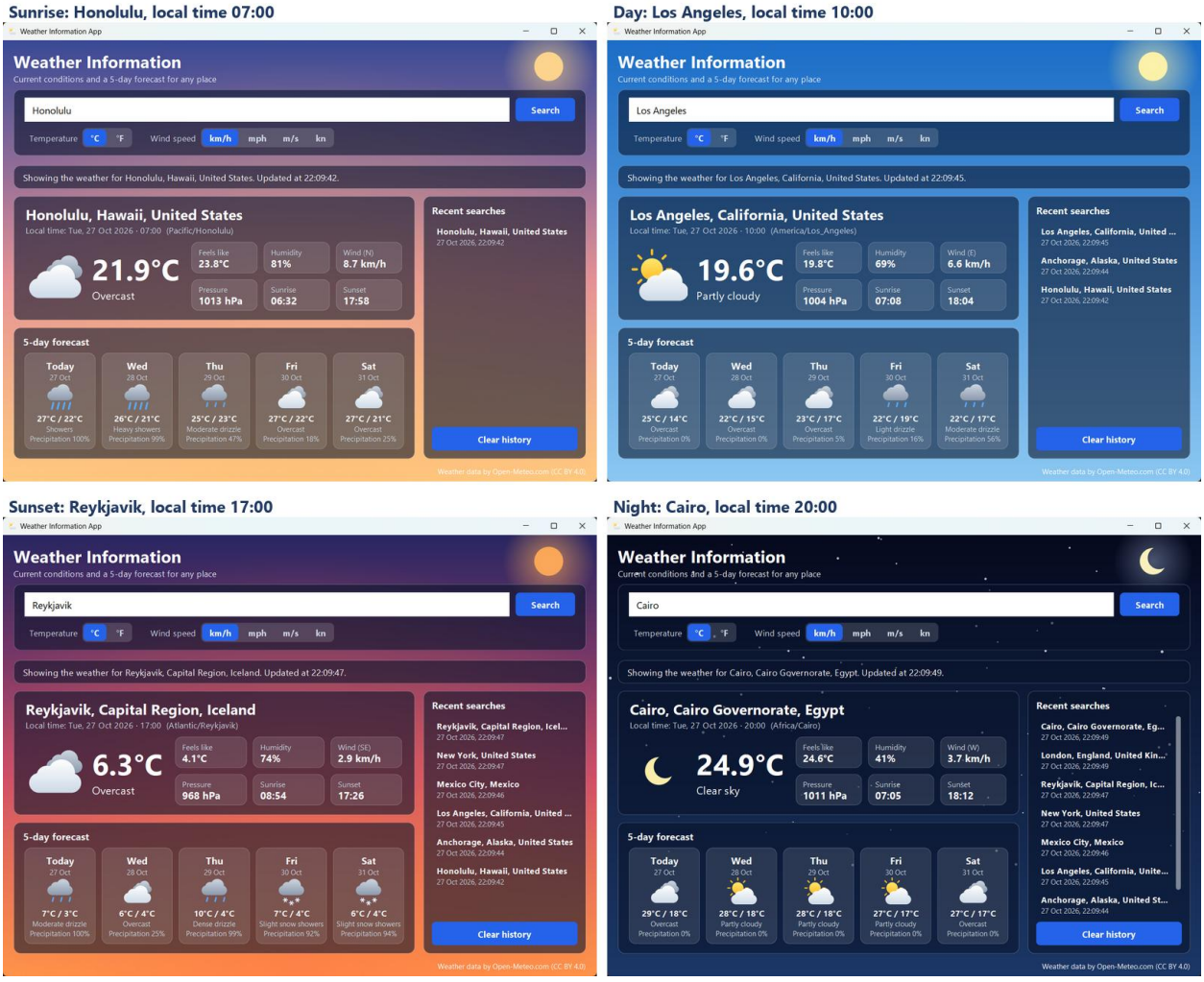
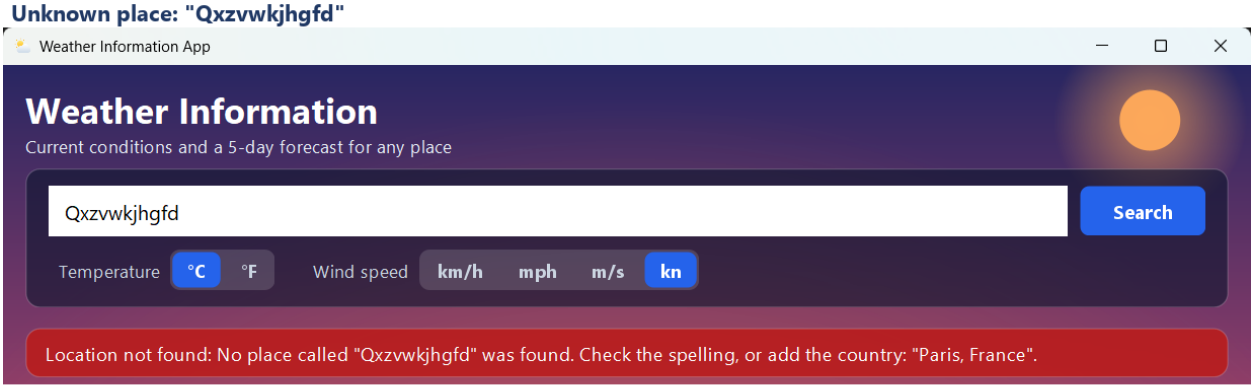
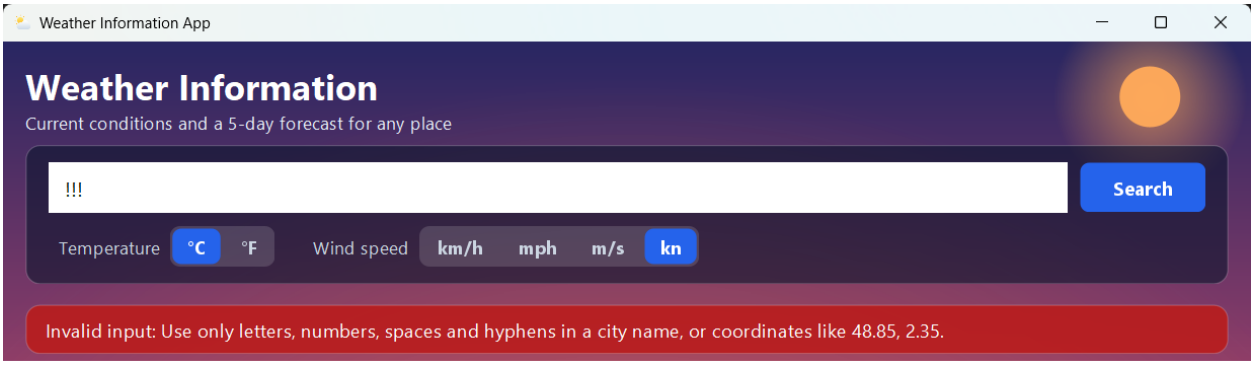


Figure 5. The dynamic background for the four parts of the day, each captured from a real search: a glowing sunrise sky, a blue day sky, a red and orange sunset, and a night sky with the moon and stars. The phase is computed from the sunrise and sunset of the searched place.

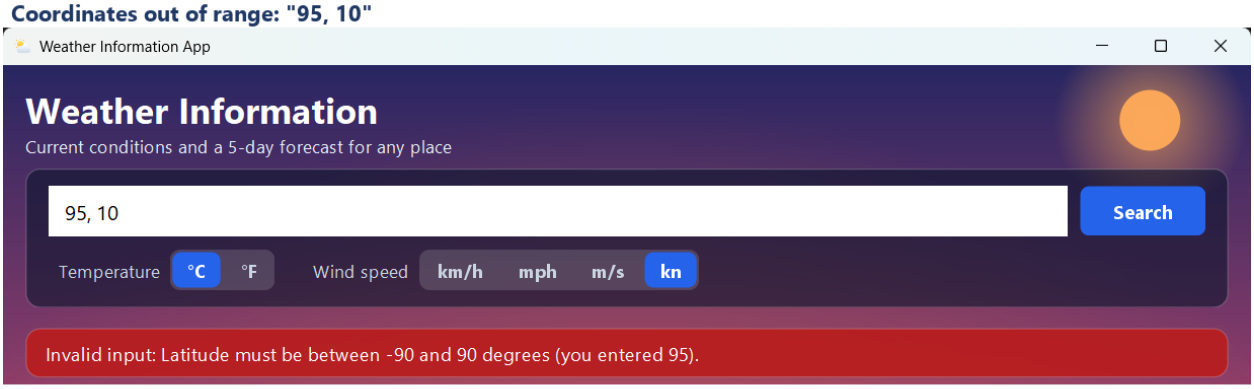
Unknown place: "Qxzvwkjhgfd"



Unusable characters: "!!!"



Coordinates out of range: "95, 10"



No connection to the weather service

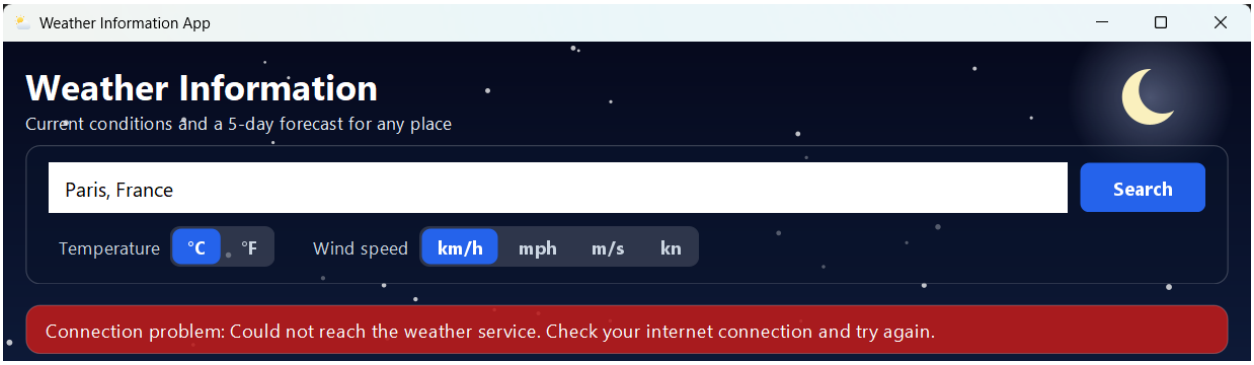


Figure 6. Error handling: an unknown place, characters that cannot be part of a place name, coordinates out of range, and no connection to the weather service. The message appears in a red bar; the previous result stays on screen and the window stays usable.

4. Testing

`tests/weatherapp/TestRunner.java` runs the automated checks and needs no test library. Its results are saved in `tests/output/test_results.txt`. The checks are grouped as follows.

Group	What is checked	Passed
Unit conversion	°C to °F; km/h to mph, m/s and knots.	9/9
Display formatting	Rounding, the "-0" case, compass points, dates and timestamps.	18/18
Weather codes and icons	Code to text and icon; unknown codes; every icon draws by day and night.	13/13
Time of day (dynamic background)	Sunrise, day, sunset and night at the boundary minutes; clock fallback.	19/19
JSON reader	Values, nesting, escapes, safe lookups, 13 kinds of malformed input.	28/28
Input validation	Cities in many alphabets, coordinates, empty and unusable input, limits.	29/29
Search history	Order, capacity, timestamps, saving, reloading, damaged lines.	13/13
Weather service (local mock server)	Parsing of every field, request parameters, URL encoding, HTTP 400/429/500, bad replies.	48/48
Network failures	Slow server (timeout), refused connection, unknown host.	6/6
Window (against the mock server)	Searches, error messages, history, units, backgrounds, data shown on screen.	37/37
Live Open-Meteo API	Real searches (London, Paris, Tashkent, coordinates) and a made-up place.	20/20

Summary: 240/240 checks passed.

How strong are the checks? As a sanity test, three bugs were introduced on purpose (Fahrenheit offset 31 instead of 32, no night phase, a wider latitude limit). 22 checks failed, so the suite does detect such mistakes.

5. Source code

All classes are in the package `weatherapp` (folder `src/weatherapp`). Compile with `javac -encoding UTF-8 -d out src/weatherapp/*.java` and start with `java -cp out weatherapp.WeatherApp`.

WeatherApp.java

```
package weatherapp;

import java.nio.file.Path;

import javax.swing.SwingUtilities;
import javax.swing.UIManager;

/**
 * Entry point of the Weather Information App.
 *
 * <p>Usage: {@code java -cp out weatherapp.WeatherApp}. The search history is stored in
 * {@code .weather-information-app/history.tsv} in the user's home folder; the system property
 * {@code weatherapp.history} can point to a different file.
 */
public final class WeatherApp {

    private static final String HISTORY_PROPERTY = "weatherapp.history";
    private static final int HISTORY_CAPACITY = 20;

    private WeatherApp() {
    }

    public static void main(String[] args) {
        // Swing components must be created and used on the event dispatch thread.
        SwingUtilities.invokeLater(() -> {
            useNativeLookAndFeel();
            SearchHistory history = new SearchHistory(historyFile(), HISTORY_CAPACITY);
            new WeatherFrame(new WeatherService(), history).setVisible(true);
        });
    }

    private static Path historyFile() {
        String custom = System.getProperty(HISTORY_PROPERTY);
        if (custom != null && !custom.isBlank()) {
            return Path.of(custom);
        }
        return Path.of(System.getProperty("user.home"), ".weather-information-app",
                "history.tsv");
    }

    static void useNativeLookAndFeel() {
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception e) {
            // The default look and feel works too, so a failure here is harmless.
        }
    }
}
```

WeatherFrame.java

```
package weatherapp;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Cursor;
import java.awt.Desktop;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.awt.event.MouseAdapter;
```

```
import java.awt.event.MouseEvent;
import java.io.IOException;
import java.net.URI;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;
import java.util.Locale;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;

import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.SwingWorker;

/**
 * The main window. It owns the controls, starts the searches and passes the results on to
 * the panels that display them.
 *
 * <p>Searches run on a background thread ({@link SwingWorker}) so the window never freezes
 * while the network is slow; the result is shown back on the event thread. The latest
 * report is kept in memory, so switching units just redraws it without asking the service
 * again.
 */
public final class WeatherFrame extends JFrame {

    private static final String ATTRIBUTION_URL = "https://open-meteo.com/";

    private final WeatherService service;
    private final SearchHistory history;

    private final BackgroundPanel background = new BackgroundPanel();
    private final HintTextField searchField =
            new HintTextField("City or coordinates, e.g. Paris, France  or  48.85, 2.35", 30);
    private final AccentButton searchButton = new AccentButton("Search");
    private final StatusBar statusBar = new StatusBar();
    private final CurrentWeatherPanel currentPanel = new CurrentWeatherPanel();
    private final ForecastPanel forecastPanel = new ForecastPanel();
    private final HistoryPanel historyPanel =
            new HistoryPanel(this::lookUpAgain, this::clearHistory);

    private TemperatureUnit temperatureUnit = TemperatureUnit.CELSIUS;
    private WindSpeedUnit windUnit = WindSpeedUnit.KILOMETRES_PER_HOUR;

    private final SegmentedControl<TemperatureUnit> temperatureControl = new SegmentedControl<>(
            List.of(TemperatureUnit.values()), TemperatureUnit::symbol, unit -> {
                temperatureUnit = unit;
                redraw();
            });
    private final SegmentedControl<WindSpeedUnit> windControl = new SegmentedControl<>(
            List.of(WindSpeedUnit.values()), WindSpeedUnit::symbol, unit -> {
                windUnit = unit;
                redraw();
            });

    /** The latest successful result; null until the first search succeeds. */
    private WeatherReport report;
    private boolean busy;

    public WeatherFrame(WeatherService service, SearchHistory history) {
        super("Weather Information App");
        this.service = service;
        this.history = history;

        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setIconImage(new WeatherIcon(IconType.PARTLY_CLOUDY, false, 64).toImage());
        setContentPane(background);
        background.setLayout(new BorderLayout(0, 12));
        background.setBorder(BorderFactory.createEmptyBorder(16, 18, 10, 18));

        background.add(buildControls(), BorderLayout.NORTH);
        background.add(buildCenter(), BorderLayout.CENTER);
```

```
        background.add(buildFooter(), BorderLayout.SOUTH);

        searchButton.setMnemonic('S');
        searchButton.setToolTipText("Look up the weather (Enter)");
        searchField.setToolTipText("Type a city name, or latitude and longitude");
        searchField.addActionListener(e -> submitSearch());
        searchButton.addActionListener(e -> submitSearch());
        getRootPane().setDefaultButton(searchButton);

        background.setTimeOfDay(TimeOfDay.fromClock(LocalDateTime.now().toLocalTime()));
        historyPanel.refresh(history.entries());
        statusBar.showInfo("Enter a city name or coordinates and press Search.");

        setMinimumSize(new Dimension(960, 740));
        setSize(1000, 770);
        setLocationRelativeTo(null);
    }

    // ---- building the window -------------------------------------------------------

    private JPanel buildControls() {
        JPanel controls = new RoundedPanel(new GridBagLayout(), Ui.CARD);
        controls.setBorder(BorderFactory.createEmptyBorder(14, 18, 14, 18));
        GridBagConstraints c = new GridBagConstraints();
        c.insets = new Insets(0, 0, 0, 10);
        c.anchor = GridBagConstraints.WEST;

        c.gridx = 0;
        c.gridy = 0;
        c.weightx = 1;
        c.fill = GridBagConstraints.HORIZONTAL;
        c.insets = new Insets(0, 0, 10, 10);
        controls.add(searchField, c);

        c.gridx = 1;
        c.weightx = 0;
        c.fill = GridBagConstraints.NONE;
        c.insets = new Insets(0, 0, 10, 0);
        controls.add(searchButton, c);

        JPanel units = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
        units.setOpaque(false);
        units.add(Ui.label("Temperature", Font.PLAIN, 14, Ui.TEXT_MUTED));
        units.add(temperatureControl);
        units.add(Box.createHorizontalStrut(14));
        units.add(Ui.label("Wind speed", Font.PLAIN, 14, Ui.TEXT_MUTED));
        units.add(windControl);

        c.gridx = 0;
        c.gridy = 1;
        c.gridwidth = 2;
        c.insets = new Insets(0, 0, 0, 0);
        controls.add(units, c);

        JPanel heading = new JPanel();
        heading.setOpaque(false);
        heading.setLayout(new BoxLayout(heading, BoxLayout.Y_AXIS));
        heading.add(Ui.label("Weather Information", Font.BOLD, 28, Ui.TEXT));
        heading.add(Ui.label("Current conditions and a 5-day forecast for any place",
                Font.PLAIN, 14, new Color(232, 238, 248)));

        JPanel top = new JPanel(new BorderLayout(0, 8));
        top.setOpaque(false);
        JPanel stack = new JPanel(new BorderLayout(0, 8));
        stack.setOpaque(false);
        stack.add(controls, BorderLayout.NORTH);
        stack.add(statusBar, BorderLayout.SOUTH);
        top.add(heading, BorderLayout.NORTH);
        top.add(stack, BorderLayout.CENTER);
        return top;
    }

    private JPanel buildCenter() {
        JPanel main = new JPanel(new GridBagLayout());
        main.setOpaque(false);
```

```
        GridBagConstraints c = new GridBagConstraints();
        c.gridx = 0;
        c.weightx = 1;
        c.fill = GridBagConstraints.BOTH;

        c.gridy = 0;
        c.weighty = 1;
        c.insets = new Insets(0, 0, 12, 0);
        main.add(currentPanel, c);

        c.gridy = 1;
        c.weighty = 0;
        c.insets = new Insets(0, 0, 0, 0);
        main.add(forecastPanel, c);

        JPanel center = new JPanel(new BorderLayout(14, 0));
        center.setOpaque(false);
        center.add(main, BorderLayout.CENTER);
        center.add(historyPanel, BorderLayout.EAST);
        return center;
    }

    private JPanel buildFooter() {
        JLabel credit = Ui.label("Weather data by Open-Meteo.com (CC BY 4.0)", Font.PLAIN, 12,
                Ui.TEXT);
        credit.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        credit.setToolTipText("Open " + ATTRIBUTION_URL);
        credit.addMouseListener(new MouseAdapter() {
            @Override
            public void mouseClicked(MouseEvent e) {
                openInBrowser(ATTRIBUTION_URL);
            }
        });
        JPanel footer = new JPanel(new BorderLayout());
        footer.setOpaque(false);
        footer.add(credit, BorderLayout.EAST);
        return footer;
    }

    private static void openInBrowser(String url) {
        boolean canBrowse = Desktop.isDesktopSupported()
                && Desktop.getDesktop().isSupported(Desktop.Action.BROWSE);
        if (canBrowse) {
            try {
                Desktop.getDesktop().browse(URI.create(url));
            } catch (IOException e) {
                System.err.println("Could not open the browser: " + e.getMessage());
            }
        }
    }

    // ---- searching ----------------------------------------------------------------

    /** Validates the text in the search box and, if it is a usable location, looks it up. */
    public void submitSearch() {
        if (busy) {
            return;
        }
        LocationQuery query;
        try {
            query = InputParser.parse(searchField.getText());
        } catch (WeatherException e) {
            statusBar.showError(e);
            return;
        }
        String label = query instanceof LocationQuery.City city
                ? "\"" + city.name() + "\"" : "the coordinates";
        lookUp("Looking up " + label + "…",
                () -> service.fetchWeather(service.findLocation(query)));
    }

    /** Shows a past search again; its coordinates are already known, so no geocoding is needed. */
    private void lookUpAgain(SearchHistory.Entry entry) {
        if (busy) {
            return;
```

```
        }
        searchField.setText(entry.location().name());
        lookUp("Looking up " + entry.location().label() + " again…",
                () -> service.fetchWeather(entry.location()));
    }

    private void clearHistory() {
        history.clear();
        historyPanel.refresh(history.entries());
    }

    /** Runs a weather request off the event thread and shows its outcome when it finishes. */
    private void lookUp(String progressMessage, Callable<WeatherReport> request) {
        setBusy(true);
        statusBar.showInfo(progressMessage);
        new SwingWorker<WeatherReport, Void>() {
            @Override
            protected WeatherReport doInBackground() throws Exception {
                return request.call();
            }

            @Override
            protected void done() {
                try {
                    show(get());
                } catch (ExecutionException e) {
                    showFailure(e.getCause());
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                } finally {
                    setBusy(false);
                }
            }
        }.execute();
    }

    private void setBusy(boolean busy) {
        this.busy = busy;
        searchButton.setEnabled(!busy);
        searchButton.setText(busy ? "Searching…" : "Search");
        setCursor(Cursor.getPredefinedCursor(busy ? Cursor.WAIT_CURSOR : Cursor.DEFAULT_CURSOR));
    }

    // ---- showing results -----------------------------------------------------------

    private void show(WeatherReport newReport) {
        report = newReport;
        history.record(newReport.location(), LocalDateTime.now());
        historyPanel.refresh(history.entries());
        redraw();
        statusBar.showInfo("Showing the weather for " + newReport.location().label()
                + ". Updated at " + DateTimeFormatter.ofPattern("HH:mm:ss", Locale.ENGLISH)
                        .format(LocalDateTime.now()) + ".");
    }

    private void showFailure(Throwable cause) {
        if (cause instanceof WeatherException problem) {
            statusBar.showError(problem);
        } else {
            cause.printStackTrace();            // a bug: keep the details for the developer
            statusBar.showUnexpectedError();
        }
    }

    /** Draws the latest report in the selected units and picks the matching background. */
    private void redraw() {
        if (report == null) {
            return;
        }
        currentPanel.show(report, temperatureUnit, windUnit);
        forecastPanel.show(report.forecast(), temperatureUnit);
        background.setTimeOfDay(TimeOfDay.at(report.current().time(),
                report.today().sunrise(), report.today().sunset()));
    }
```

```
    // ---- hooks used by the tests and the screenshot tool --------------------------------

    public void setSearchText(String text) {
        searchField.setText(text);
    }

    public void chooseUnits(TemperatureUnit temperature, WindSpeedUnit wind) {
        temperatureControl.select(temperature);
        windControl.select(wind);
    }

    public boolean isBusy() {
        return busy;
    }

    public String statusText() {
        return statusBar.text();
    }

    public TimeOfDay timeOfDay() {
        return background.timeOfDay();
    }

    public int historySize() {
        return historyPanel.entryCount();
    }

    public WeatherReport currentReport() {
        return report;
    }
}
```

WeatherService.java

```
package weatherapp;

import java.io.IOException;
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpTimeoutException;
import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeParseException;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

import weatherapp.WeatherException.Type;

/**
 * Talks to the Open-Meteo web services (https://open-meteo.com).
 *
 * <p>Two services are used: the <b>geocoding API</b> turns a place name into coordinates, and
 * the <b>forecast API</b> returns the current weather and a daily forecast for coordinates.
 * Both are free for non-commercial use and need no API key. Every call is a plain HTTPS GET
 * that returns JSON; one search costs at most two calls, and changing the display units
 * never calls the service again because values are converted locally.
 *
 * <p>The methods block while they wait for the network, so the GUI calls them from a
 * background thread. Every failure is reported as a {@link WeatherException} with a message
 * that is safe to show to the user.
 */
public class WeatherService {

    public static final String GEOCODING_URL = "https://geocoding-api.open-meteo.com/v1/search";
    public static final String FORECAST_URL = "https://api.open-meteo.com/v1/forecast";

    /** How many days the forecast covers, today included. */
    public static final int FORECAST_DAYS = 5;
```

```
private static final String CURRENT_FIELDS = "temperature_2m,relative_humidity_2m,"
        + "apparent_temperature,is_day,weather_code,surface_pressure,"
        + "wind_speed_10m,wind_direction_10m";
private static final String DAILY_FIELDS = "weather_code,temperature_2m_max,"
        + "temperature_2m_min,precipitation_probability_max,sunrise,sunset";
private static final String USER_AGENT = "WeatherInformationApp/1.0 (student project)";

private final HttpClient client;
private final String geocodingUrl;
private final String forecastUrl;
private final Duration requestTimeout;

/** A service that uses the real Open-Meteo servers. */
public WeatherService() {
    this(GEOCODING_URL, FORECAST_URL, Duration.ofSeconds(10));
}

/** A service with custom addresses and timeout; the tests use this with a local server. */
public WeatherService(String geocodingUrl, String forecastUrl, Duration requestTimeout) {
    this.geocodingUrl = geocodingUrl;
    this.forecastUrl = forecastUrl;
    this.requestTimeout = requestTimeout;
    this.client = HttpClient.newBuilder().connectTimeout(requestTimeout).build();
}

/**
 * Resolves what the user typed to a place. Coordinates need no lookup; a city name is
 * sent to the geocoding API, which returns the best match first.
 *
 * @throws WeatherException NOT_FOUND if no place matches, NETWORK or SERVICE on failures
 */
public Location findLocation(LocationQuery query) throws WeatherException {
    if (query instanceof LocationQuery.Coordinates coordinates) {
        return Location.ofCoordinates(coordinates.latitude(), coordinates.longitude());
    }
    String name = ((LocationQuery.City) query).name();
    Json response = getJson(geocodingUrl + "?name=" + encode(name)
            + "&count=1&language=en&format=json");

    Json best = response.get("results").get(0);
    if (best.isMissing()) {
        throw new WeatherException(Type.NOT_FOUND, "No place called \"" + name
                + "\" was found. Check the spelling, or add the country: \"Paris, France\".");
    }
    double latitude = best.get("latitude").asDouble(Double.NaN);
    double longitude = best.get("longitude").asDouble(Double.NaN);
    if (Double.isNaN(latitude) || Double.isNaN(longitude)) {
        throw incompleteData();
    }
    return new Location(best.get("name").asString(name), best.get("admin1").asString(""),
            best.get("country").asString(""), latitude, longitude);
}

/**
 * Downloads the current weather and the daily forecast for a place.
 *
 * @throws WeatherException NETWORK or SERVICE if the data cannot be retrieved or read
 */
public WeatherReport fetchWeather(Location location) throws WeatherException {
    String url = String.format(Locale.ROOT,
            "%s?latitude=%.4f&longitude=%.4f&current=%s&daily=%s&timezone=auto"
                    + "&forecast_days=%d",
            forecastUrl, location.latitude(), location.longitude(),
            CURRENT_FIELDS, DAILY_FIELDS, FORECAST_DAYS);
    Json root = getJson(url);
    try {
        return new WeatherReport(location, root.get("timezone").asString(""),
                readCurrent(root.get("current")), readForecast(root.get("daily")));
    } catch (DateTimeParseException e) {
        throw new WeatherException(Type.SERVICE,
                "The weather service sent a date the app could not read.", e);
    }
}
```

```
    private static CurrentConditions readCurrent(Json current) throws WeatherException {
        return new CurrentConditions(
                LocalDateTime.parse(text(current, "time")),
                number(current, "temperature_2m"),
                number(current, "apparent_temperature"),
                (int) Math.round(number(current, "relative_humidity_2m")),
                number(current, "wind_speed_10m"),
                (int) Math.round(number(current, "wind_direction_10m")),
                number(current, "surface_pressure"),
                (int) Math.round(number(current, "weather_code")),
                number(current, "is_day") == 1);
    }

    private static List<DailyForecast> readForecast(Json daily) throws WeatherException {
        int days = daily.get("time").size();
        if (days == 0) {
            throw incompleteData();
        }
        List<DailyForecast> forecast = new ArrayList<>();
        for (int i = 0; i < days; i++) {
            String sunrise = daily.get("sunrise").get(i).asString(null);
            String sunset = daily.get("sunset").get(i).asString(null);
            forecast.add(new DailyForecast(
                    LocalDate.parse(daily.get("time").get(i).asString("")),
                    (int) Math.round(number(daily.get("weather_code"), i)),
                    number(daily.get("temperature_2m_max"), i),
                    number(daily.get("temperature_2m_min"), i),
                    daily.get("precipitation_probability_max").get(i)
                            .asInt(DailyForecast.UNKNOWN_CHANCE),
                    sunrise == null ? null : LocalDateTime.parse(sunrise),
                    sunset == null ? null : LocalDateTime.parse(sunset)));
        }
        return forecast;
    }

    /** A required numeric property; throws if the service left it out. */
    private static double number(Json object, String property) throws WeatherException {
        Json value = object.get(property);
        if (!value.isNumber()) {
            throw incompleteData();
        }
        return value.asDouble(0);
    }

    /** A required numeric array element; throws if the service left it out. */
    private static double number(Json array, int index) throws WeatherException {
        Json value = array.get(index);
        if (!value.isNumber()) {
            throw incompleteData();
        }
        return value.asDouble(0);
    }

    private static String text(Json object, String property) throws WeatherException {
        String value = object.get(property).asString(null);
        if (value == null) {
            throw incompleteData();
        }
        return value;
    }

    /** Sends a GET request and returns the parsed JSON reply, or throws a friendly error. */
    private Json getJson(String url) throws WeatherException {
        HttpRequest request = HttpRequest.newBuilder(URI.create(url))
                .timeout(requestTimeout)
                .header("User-Agent", USER_AGENT)
                .header("Accept", "application/json")
                .GET()
                .build();

        HttpResponse<String> response;
        try {
            response = client.send(request, HttpResponse.BodyHandlers.ofString());
        } catch (HttpTimeoutException e) {
            throw new WeatherException(Type.NETWORK,
```

```
                    "The weather service took too long to answer. Please try again.", e);
        } catch (IOException e) {
            throw new WeatherException(Type.NETWORK, "Could not reach the weather service. "
                    + "Check your internet connection and try again.", e);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new WeatherException(Type.NETWORK, "The request was interrupted.", e);
        }

        int status = response.statusCode();
        Json body;
        try {
            body = Json.parse(response.body());
        } catch (IllegalArgumentException e) {
            if (status >= 400) {
                throw httpError(status, Json.missing());
            }
            throw new WeatherException(Type.SERVICE,
                    "The weather service sent a reply the app could not read.", e);
        }
        if (status >= 400 || body.get("error").asBoolean(false)) {
            throw httpError(status, body);
        }
        return body;
    }

    private static WeatherException httpError(int status, Json body) {
        if (status == 429) {
            return new WeatherException(Type.SERVICE, "The weather service is receiving too "
                    + "many requests. Wait a minute and try again.");
        }
        String reason = body.get("reason").asString("HTTP status " + status);
        return new WeatherException(Type.SERVICE,
                "The weather service could not answer the request: " + reason);
    }

    private static WeatherException incompleteData() {
        return new WeatherException(Type.SERVICE,
                "The weather service returned incomplete data. Please try again later.");
    }

    private static String encode(String text) {
        return URLEncoder.encode(text, StandardCharsets.UTF_8);
    }
}
```

Json.java

```
package weatherapp;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * A small, read-only JSON reader.
 *
 * <p>The app only needs to read the replies of the weather API, so a tiny parser keeps the
 * project free of external libraries (it compiles and runs with a plain JDK). A parsed value
 * is navigated with {@link #get(String)} and {@link #get(int)}. Asking for something that
 * is not there never throws; it returns a "missing" value, which makes chained lookups such
 * as {@code root.get("current").get("temperature_2m")} safe. A JSON {@code null} counts as
 * missing too, because that is how the API reports data it does not have.
 */
public final class Json {

    private static final Json MISSING = new Json(null);
    private static final int MAX_DEPTH = 64;

    /** A Map, List, String, Double, Boolean, or null. */
    private final Object value;

    private Json(Object value) {
```

```
        this.value = value;
    }

    /** Parses a complete JSON document; throws IllegalArgumentException if it is malformed. */
    public static Json parse(String text) {
        Parser parser = new Parser(text);
        Object result = parser.readValue(0);
        parser.skipWhitespace();
        if (!parser.atEnd()) {
            throw parser.error("unexpected text after the end of the JSON value");
        }
        return new Json(result);
    }

    /** The value used when something is absent. */
    public static Json missing() {
        return MISSING;
    }

    /** The property with the given name, or a missing value. */
    public Json get(String key) {
        if (value instanceof Map<?, ?> map && map.containsKey(key)) {
            return new Json(map.get(key));
        }
        return MISSING;
    }

    /** The array element at the given index, or a missing value. */
    public Json get(int index) {
        if (value instanceof List<?> list && index >= 0 && index < list.size()) {
            return new Json(list.get(index));
        }
        return MISSING;
    }

    /** The number of elements of an array or properties of an object (0 for anything else). */
    public int size() {
        if (value instanceof List<?> list) {
            return list.size();
        }
        if (value instanceof Map<?, ?> map) {
            return map.size();
        }
        return 0;
    }

    public boolean isMissing() {
        return value == null;
    }

    public boolean isNumber() {
        return value instanceof Double;
    }

    public double asDouble(double fallback) {
        return value instanceof Double number ? number : fallback;
    }

    public int asInt(int fallback) {
        return value instanceof Double number ? (int) Math.round(number) : fallback;
    }

    public String asString(String fallback) {
        return value instanceof String text ? text : fallback;
    }

    public boolean asBoolean(boolean fallback) {
        return value instanceof Boolean flag ? flag : fallback;
    }

    /** Recursive-descent parser for the JSON grammar (RFC 8259). */
    private static final class Parser {
        private final String text;
        private int pos;
```

```
        Parser(String text) {
            this.text = text;
        }

        boolean atEnd() {
            return pos >= text.length();
        }

        IllegalArgumentException error(String problem) {
            return new IllegalArgumentException("Invalid JSON at position " + pos + ": " + problem);
        }

        void skipWhitespace() {
            while (!atEnd() && " \t\n\r".indexOf(text.charAt(pos)) >= 0) {
                pos++;
            }
        }

        /** The current character, or '\0' at the end of the input. */
        private char peek() {
            return atEnd() ? '\0' : text.charAt(pos);
        }

        private void expect(char expected) {
            if (peek() != expected) {
                throw error("expected '" + expected + "'");
            }
            pos++;
        }

        Object readValue(int depth) {
            if (depth > MAX_DEPTH) {
                throw error("nesting is too deep");
            }
            skipWhitespace();
            return switch (peek()) {
                case '{' -> readObject(depth);
                case '[' -> readArray(depth);
                case '"' -> readString();
                case 't' -> readWord("true", Boolean.TRUE);
                case 'f' -> readWord("false", Boolean.FALSE);
                case 'n' -> readWord("null", null);
                default -> readNumber();
            };
        }

        private Object readWord(String word, Object result) {
            if (!text.startsWith(word, pos)) {
                throw error("unexpected character");
            }
            pos += word.length();
            return result;
        }

        private Map<String, Object> readObject(int depth) {
            Map<String, Object> map = new LinkedHashMap<>();
            pos++;                                      // the opening '{'
            skipWhitespace();
            if (peek() == '}') {
                pos++;
                return map;
            }
            while (true) {
                skipWhitespace();
                if (peek() != '"') {
                    throw error("expected a quoted property name");
                }
                String key = readString();
                skipWhitespace();
                expect(':');
                map.put(key, readValue(depth + 1));
                skipWhitespace();
                if (peek() == ',') {
                    pos++;
                } else {
```

```
            expect('}');
            return map;
        }
    }
}

private List<Object> readArray(int depth) {
    List<Object> list = new ArrayList<>();
    pos++;                                  // the opening '['
    skipWhitespace();
    if (peek() == ']') {
        pos++;
        return list;
    }
    while (true) {
        list.add(readValue(depth + 1));
        skipWhitespace();
        if (peek() == ',') {
            pos++;
        } else {
            expect(']');
            return list;
        }
    }
}

private String readString() {
    pos++;                                  // the opening quote
    StringBuilder out = new StringBuilder();
    while (true) {
        if (atEnd()) {
            throw error("the string is not closed");
        }
        char c = text.charAt(pos++);
        if (c == '"') {
            return out.toString();
        }
        if (c != '\\') {
            out.append(c);
            continue;
        }
        if (atEnd()) {
            throw error("the string is not closed");
        }
        char escape = text.charAt(pos++);
        switch (escape) {
            case '"', '\\', '/' -> out.append(escape);
            case 'b' -> out.append('\b');
            case 'f' -> out.append('\f');
            case 'n' -> out.append('\n');
            case 'r' -> out.append('\r');
            case 't' -> out.append('\t');
            case 'u' -> out.append(readUnicodeEscape());
            default -> throw error("unknown escape sequence '\\" + escape + "'");
        }
    }
}

private char readUnicodeEscape() {
    if (pos + 4 > text.length()) {
        throw error("incomplete \\u escape");
    }
    try {
        char c = (char) Integer.parseInt(text.substring(pos, pos + 4), 16);
        pos += 4;
        return c;
    } catch (NumberFormatException e) {
        throw error("bad \\u escape");
    }
}

private Double readNumber() {
    int start = pos;
    while (!atEnd() && "+-0123456789.eE".indexOf(text.charAt(pos)) >= 0) {
        pos++;
```

```
            }
            if (start == pos) {
                throw error("unexpected character");
            }
            try {
                return Double.parseDouble(text.substring(start, pos));
            } catch (NumberFormatException e) {
                pos = start;
                throw error("malformed number");
            }
        }
    }
}
```

InputParser.java

```
package weatherapp;

import java.util.Locale;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 * Checks what the user typed in the search box and turns it into a {@link LocationQuery}.
 *
 * <p>Validating here means that obviously wrong input is rejected immediately, with a helpful
 * message, instead of being sent to the weather service.
 */
public final class InputParser {

    private static final int MAX_LENGTH = 100;

    /** Two decimal numbers separated by a comma, a semicolon or a space: "48.85, 2.35". */
    private static final Pattern COORDINATES = Pattern.compile(
            "^([+-]?\\d{1,3}(?:\\.\\d+)?)\\s*(?:[,;]|\\s)\\s*([+-]?\\d{1,3}(?:\\.\\d+)?)$");

    /** A single number such as "48.85", which is neither a place nor a coordinate pair. */
    private static final Pattern SINGLE_NUMBER = Pattern.compile("^[+-]?\\d+(?:[.,]\\d+)?$");

    /** Letters (any alphabet), digits, spaces and the punctuation used in place names. */
    private static final Pattern PLACE_NAME =
            Pattern.compile("^[\\p{L}\\p{M}\\d][\\p{L}\\p{M}\\d\\s'’.,()\\-]*$");

    private static final String COORDINATE_HINT =
            "For coordinates, enter latitude and longitude, for example 48.85, 2.35.";

    private InputParser() {
    }

    /**
     * Validates the text and classifies it.
     *
     * @throws WeatherException of type INVALID_INPUT if the text cannot be a location
     */
    public static LocationQuery parse(String input) throws WeatherException {
        String text = input == null ? "" : input.trim().replaceAll("\\s+", " ");
        if (text.isEmpty()) {
            throw invalid("Please enter a city name or coordinates first.");
        }
        if (text.length() > MAX_LENGTH) {
            throw invalid("That is too long for a location. Use at most " + MAX_LENGTH
                    + " characters.");
        }

        Matcher coordinates = COORDINATES.matcher(text);
        if (coordinates.matches()) {
            return parseCoordinates(coordinates);
        }
        if (SINGLE_NUMBER.matcher(text).matches()) {
            throw invalid("A single number is not a place. " + COORDINATE_HINT);
        }
        if (!PLACE_NAME.matcher(text).matches()) {
            throw invalid("Use only letters, numbers, spaces and hyphens in a city name, "
                    + "or coordinates like 48.85, 2.35.");
```

```
        }
        if (text.codePoints().filter(Character::isLetterOrDigit).count() < 2) {
            throw invalid("A city name needs at least two letters.");
        }
        return new LocationQuery.City(text);
    }

    private static LocationQuery parseCoordinates(Matcher matcher) throws WeatherException {
        double latitude = Double.parseDouble(matcher.group(1));
        double longitude = Double.parseDouble(matcher.group(2));
        if (latitude < -90 || latitude > 90) {
            throw invalid(String.format(Locale.ENGLISH,
                    "Latitude must be between -90 and 90 degrees (you entered %s).",
                    matcher.group(1)));
        }
        if (longitude < -180 || longitude > 180) {
            throw invalid(String.format(Locale.ENGLISH,
                    "Longitude must be between -180 and 180 degrees (you entered %s).",
                    matcher.group(2)));
        }
        return new LocationQuery.Coordinates(latitude, longitude);
    }

    private static WeatherException invalid(String message) {
        return new WeatherException(WeatherException.Type.INVALID_INPUT, message);
    }
}
```

LocationQuery.java

```
package weatherapp;

/** What the user asked for: either a place name or a pair of coordinates. */
public sealed interface LocationQuery {

    /** A place name such as "Paris" or "Paris, France". */
    record City(String name) implements LocationQuery { }

    /** A latitude and longitude in decimal degrees. */
    record Coordinates(double latitude, double longitude) implements LocationQuery { }
}
```

Location.java

```
package weatherapp;

import java.util.Locale;
import java.util.Objects;

/**
 * A place on Earth: where the weather is asked for.
 *
 * @param name      the place name, or the formatted coordinates for a coordinate search
 * @param region    the state or province (may be empty)
 * @param country   the country (may be empty)
 * @param latitude  degrees north (negative is south)
 * @param longitude degrees east (negative is west)
 */
public record Location(String name, String region, String country,
                       double latitude, double longitude) {

    public Location {
        name = Objects.requireNonNullElse(name, "");
        region = Objects.requireNonNullElse(region, "");
        country = Objects.requireNonNullElse(country, "");
    }

    /** A location known only by its coordinates, named after them. */
    public static Location ofCoordinates(double latitude, double longitude) {
        String name = String.format(Locale.ENGLISH, "%.2f°%s, %.2f°%s",
                Math.abs(latitude), latitude >= 0 ? "N" : "S",
                Math.abs(longitude), longitude >= 0 ? "E" : "W");
        return new Location(name, "", "", latitude, longitude);
    }
```

```
    /** Text for the screen and the history, such as "Paris, Île-de-France, France". */
    public String label() {
        StringBuilder label = new StringBuilder(name);
        if (!region.isBlank() && !region.equalsIgnoreCase(name)) {
            label.append(", ").append(region);
        }
        if (!country.isBlank()) {
            label.append(", ").append(country);
        }
        return label.toString();
    }
}
```

CurrentConditions.java

```
package weatherapp;

import java.time.LocalDateTime;

/**
 * The weather right now at a location. Temperatures are in degrees Celsius and the wind
 * speed in km/h; {@link TemperatureUnit} and {@link WindSpeedUnit} convert them for display.
 *
 * @param time                the observation time, in the location's own time zone
 * @param temperatureC        air temperature
 * @param feelsLikeC          apparent ("feels like") temperature
 * @param humidityPercent     relative humidity, 0 to 100
 * @param windSpeedKmh        wind speed
 * @param windDirectionDegrees where the wind comes from, 0 to 360 (0 is north)
 * @param pressureHpa         air pressure at the surface
 * @param weatherCode         the API's weather code, see {@link WeatherCode}
 * @param daytime             true if the sun is up, which chooses the sun or moon icon
 */
public record CurrentConditions(LocalDateTime time, double temperatureC, double feelsLikeC,
                                int humidityPercent, double windSpeedKmh,
                                int windDirectionDegrees, double pressureHpa,
                                int weatherCode, boolean daytime) {
}
```

DailyForecast.java

```
package weatherapp;

import java.time.LocalDate;
import java.time.LocalDateTime;

/**
 * The forecast for one day, in the location's own time zone.
 *
 * @param date                      the day
 * @param weatherCode               the API's weather code, see {@link WeatherCode}
 * @param highC                     the highest temperature, in degrees Celsius
 * @param lowC                      the lowest temperature, in degrees Celsius
 * @param precipitationChancePercent the chance of rain or snow, or -1 if unknown
 * @param sunrise                   the time of sunrise, or null if unknown
 * @param sunset                    the time of sunset, or null if unknown
 */
public record DailyForecast(LocalDate date, int weatherCode, double highC, double lowC,
                            int precipitationChancePercent,
                            LocalDateTime sunrise, LocalDateTime sunset) {

    /** Marks a precipitation chance that the service did not provide. */
    public static final int UNKNOWN_CHANCE = -1;
}
```

WeatherReport.java

```
package weatherapp;

import java.util.List;

/**
```

```
 * Everything the app shows for one search: the place, the weather now and the forecast.
 *
 * @param location the place the report is about
 * @param timezone the place's time zone name, such as "Europe/London"
 * @param current  the weather right now
 * @param forecast the daily forecast; the first entry is today
 */
public record WeatherReport(Location location, String timezone, CurrentConditions current,
                            List<DailyForecast> forecast) {

    public WeatherReport {
        forecast = List.copyOf(forecast);
    }

    /** Today's forecast, which holds today's sunrise and sunset. */
    public DailyForecast today() {
        return forecast.get(0);
    }
}
```

WeatherException.java

```
package weatherapp;

/**
 * A problem that can be explained to the user in plain language.
 *
 * <p>Every failure the app expects (bad input, an unknown place, no internet connection,
 * a misbehaving weather service) is reported with this one checked exception, so the
 * GUI has a single place where it turns errors into messages.
 */
public class WeatherException extends Exception {

    /** The kind of problem; the GUI uses it to choose a short title for the message. */
    public enum Type {
        INVALID_INPUT("Invalid input"),
        NOT_FOUND("Location not found"),
        NETWORK("Connection problem"),
        SERVICE("Weather service problem");

        private final String title;

        Type(String title) {
            this.title = title;
        }

        public String title() {
            return title;
        }
    }

    private final Type type;

    public WeatherException(Type type, String message) {
        super(message);
        this.type = type;
    }

    public WeatherException(Type type, String message, Throwable cause) {
        super(message, cause);
        this.type = type;
    }

    public Type type() {
        return type;
    }
}
```

TemperatureUnit.java

```
package weatherapp;

/**
 * Units the temperature can be shown in. The app stores every temperature in Celsius
```

```
 * (the unit the weather service returns) and converts only when it is displayed.
 */
public enum TemperatureUnit {
    CELSIUS("°C") {
        @Override
        public double fromCelsius(double celsius) {
            return celsius;
        }
    },
    FAHRENHEIT("°F") {
        @Override
        public double fromCelsius(double celsius) {
            return celsius * 9.0 / 5.0 + 32.0;
        }
    };

    private final String symbol;

    TemperatureUnit(String symbol) {
        this.symbol = symbol;
    }

    /** Converts a temperature given in degrees Celsius to this unit. */
    public abstract double fromCelsius(double celsius);

    public String symbol() {
        return symbol;
    }
}
```

WindSpeedUnit.java

```
package weatherapp;

/**
 * Units the wind speed can be shown in. Speeds are stored in kilometres per hour (the unit
 * the weather service returns) and converted only when they are displayed.
 */
public enum WindSpeedUnit {
    KILOMETRES_PER_HOUR("km/h", 1.0),
    MILES_PER_HOUR("mph", 0.621371),
    METRES_PER_SECOND("m/s", 1.0 / 3.6),
    KNOTS("kn", 0.539957);

    private final String symbol;
    private final double perKilometrePerHour;

    WindSpeedUnit(String symbol, double perKilometrePerHour) {
        this.symbol = symbol;
        this.perKilometrePerHour = perKilometrePerHour;
    }

    /** Converts a speed given in kilometres per hour to this unit. */
    public double fromKilometresPerHour(double kmh) {
        return kmh * perKilometrePerHour;
    }

    public String symbol() {
        return symbol;
    }
}
```

DisplayFormat.java

```
package weatherapp;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Locale;

/**
 * Turns numbers and dates into the text shown in the window. All formatting uses English
 * conventions (a decimal point, English day names) so the display looks the same on every
```

```
 * computer.
 */
public final class DisplayFormat {

    private static final String[] COMPASS = {"N", "NNE", "NE", "ENE", "E", "ESE", "SE", "SSE",
            "S", "SSW", "SW", "WSW", "W", "WNW", "NW", "NNW"};

    private static final DateTimeFormatter DATE_TIME =
            DateTimeFormatter.ofPattern("EEE, d MMM yyyy '·' HH:mm", Locale.ENGLISH);
    private static final DateTimeFormatter TIME =
            DateTimeFormatter.ofPattern("HH:mm", Locale.ENGLISH);
    private static final DateTimeFormatter HISTORY_TIME =
            DateTimeFormatter.ofPattern("d MMM yyyy, HH:mm:ss", Locale.ENGLISH);
    private static final DateTimeFormatter WEEKDAY =
            DateTimeFormatter.ofPattern("EEE", Locale.ENGLISH);
    private static final DateTimeFormatter DAY_MONTH =
            DateTimeFormatter.ofPattern("d MMM", Locale.ENGLISH);

    private DisplayFormat() {
    }

    /** A temperature converted from Celsius, such as "57.7°F". */
    public static String temperature(double celsius, TemperatureUnit unit, int decimals) {
        return number(unit.fromCelsius(celsius), decimals) + unit.symbol();
    }

    /** A wind speed converted from km/h, such as "10.5 mph". */
    public static String windSpeed(double kmh, WindSpeedUnit unit) {
        return number(unit.fromKilometresPerHour(kmh), 1) + " " + unit.symbol();
    }

    /** The compass point for a wind direction in degrees, such as 261 gives "W". */
    public static String compassPoint(int degrees) {
        int normalised = ((degrees % 360) + 360) % 360;
        return COMPASS[(int) Math.round(normalised / 22.5) % COMPASS.length];
    }

    public static String percent(int value) {
        return value + "%";
    }

    public static String pressure(double hectopascals) {
        return Math.round(hectopascals) + " hPa";
    }

    /** Date and time at the searched place, such as "Tue, 27 Oct 2026 · 16:45". */
    public static String dateTime(LocalDateTime time) {
        return DATE_TIME.format(time);
    }

    /** The clock time, such as "06:45"; "--:--" when unknown. */
    public static String clockTime(LocalDateTime time) {
        return time == null ? "--:--" : TIME.format(time);
    }

    /** The timestamp shown in the history, such as "27 Oct 2026, 21:35:12". */
    public static String historyTime(LocalDateTime time) {
        return HISTORY_TIME.format(time);
    }

    /** "Today" for the current day, otherwise the weekday, such as "Wed". */
    public static String dayName(LocalDate date, LocalDate today) {
        return date.equals(today) ? "Today" : WEEKDAY.format(date);
    }

    public static String dayAndMonth(LocalDate date) {
        return DAY_MONTH.format(date);
    }

    /** Formats with a fixed number of decimals and avoids the odd result "-0". */
    private static String number(double value, int decimals) {
        String text = String.format(Locale.ENGLISH, "%." + decimals + "f", value);
        return text.matches("-0(\\.0+)?") ? text.substring(1) : text;
    }
}
```

WeatherCode.java

```
package weatherapp;

import java.util.Map;

/**
 * Translates the numeric weather codes of the API (WMO "weather interpretation codes")
 * into text and into the icon that represents them.
 */
public final class WeatherCode {

    private record Info(String description, IconType icon) { }

    private static final Info UNKNOWN = new Info("Unknown conditions", IconType.CLOUDY);

    private static final Map<Integer, Info> CODES = Map.ofEntries(
            entry(0, "Clear sky", IconType.CLEAR),
            entry(1, "Mainly clear", IconType.CLEAR),
            entry(2, "Partly cloudy", IconType.PARTLY_CLOUDY),
            entry(3, "Overcast", IconType.CLOUDY),
            entry(45, "Fog", IconType.FOG),
            entry(48, "Rime fog", IconType.FOG),
            entry(51, "Light drizzle", IconType.DRIZZLE),
            entry(53, "Moderate drizzle", IconType.DRIZZLE),
            entry(55, "Dense drizzle", IconType.DRIZZLE),
            entry(56, "Icy drizzle", IconType.DRIZZLE),
            entry(57, "Heavy icy drizzle", IconType.DRIZZLE),
            entry(61, "Slight rain", IconType.RAIN),
            entry(63, "Moderate rain", IconType.RAIN),
            entry(65, "Heavy rain", IconType.RAIN),
            entry(66, "Light freezing rain", IconType.RAIN),
            entry(67, "Heavy freezing rain", IconType.RAIN),
            entry(71, "Slight snowfall", IconType.SNOW),
            entry(73, "Moderate snowfall", IconType.SNOW),
            entry(75, "Heavy snowfall", IconType.SNOW),
            entry(77, "Snow grains", IconType.SNOW),
            entry(80, "Light showers", IconType.RAIN),
            entry(81, "Showers", IconType.RAIN),
            entry(82, "Heavy showers", IconType.RAIN),
            entry(85, "Slight snow showers", IconType.SNOW),
            entry(86, "Heavy snow showers", IconType.SNOW),
            entry(95, "Thunderstorm", IconType.THUNDERSTORM),
            entry(96, "Thunderstorm, hail", IconType.THUNDERSTORM),
            entry(99, "Thunder, heavy hail", IconType.THUNDERSTORM));

    private WeatherCode() {
    }

    private static Map.Entry<Integer, Info> entry(int code, String description, IconType icon) {
        return Map.entry(code, new Info(description, icon));
    }

    /** A short English description such as "Slight rain". */
    public static String describe(int code) {
        return CODES.getOrDefault(code, UNKNOWN).description();
    }

    /** The icon that represents the code. */
    public static IconType iconFor(int code) {
        return CODES.getOrDefault(code, UNKNOWN).icon();
    }
}
```

IconType.java

```
package weatherapp;

/** The picture used to represent a kind of weather; drawn by {@link WeatherIcon}. */
public enum IconType {
    CLEAR,
    PARTLY_CLOUDY,
    CLOUDY,
    FOG,
```

```
    DRIZZLE,
    RAIN,
    SNOW,
    THUNDERSTORM
}
```

WeatherIcon.java

```
package weatherapp;

import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Component;
import java.awt.GradientPaint;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.geom.AffineTransform;
import java.awt.geom.Area;
import java.awt.geom.Ellipse2D;
import java.awt.geom.Line2D;
import java.awt.geom.Path2D;
import java.awt.geom.RoundRectangle2D;
import java.awt.image.BufferedImage;

import javax.swing.Icon;

/**
 * A weather picture drawn with Java 2D shapes (no image files needed), so it stays sharp at
 * every size. The artwork is designed on a 100 x 100 grid and scaled to the requested size.
 */
public final class WeatherIcon implements Icon {

    private static final Color SUN = new Color(255, 190, 40);
    private static final Color SUN_LIGHT = new Color(255, 232, 110);
    private static final Color MOON = new Color(250, 236, 170);
    private static final Color DROP = new Color(90, 170, 255);
    private static final Color SNOWFLAKE = new Color(225, 240, 255);
    private static final Color BOLT = new Color(255, 214, 64);

    // Cloud colours as {top, bottom} of a vertical gradient.
    private static final Color[] CLOUD_LIGHT = {new Color(255, 255, 255), new Color(205, 218, 232)};
    private static final Color[] CLOUD_GREY = {new Color(205, 214, 225), new Color(140, 154, 172)};
    private static final Color[] CLOUD_DARK = {new Color(150, 162, 178), new Color(88, 100, 118)};

    /** The cloud is drawn in a 100 x 64 box and then moved and scaled. */
    private static final double CLOUD_HEIGHT = 64;

    private final IconType type;
    private final boolean night;
    private final int size;

    /**
     * @param type  which weather to draw
     * @param night draw the moon instead of the sun where the picture has one
     * @param size  width and height in pixels
     */
    public WeatherIcon(IconType type, boolean night, int size) {
        this.type = type;
        this.night = night;
        this.size = size;
    }

    @Override
    public int getIconWidth() {
        return size;
    }

    @Override
    public int getIconHeight() {
        return size;
    }

    @Override
    public void paintIcon(Component c, Graphics g, int x, int y) {
```

```
        Graphics2D g2 = Ui.smooth((Graphics2D) g.create());
        try {
            g2.translate(x, y);
            g2.scale(size / 100.0, size / 100.0);
            switch (type) {
                case CLEAR -> sunOrMoon(g2, 50, 50, 24);
                case PARTLY_CLOUDY -> {
                    sunOrMoon(g2, 38, 38, 19);
                    cloud(g2, 16, 38, 0.85, CLOUD_LIGHT);
                }
                case CLOUDY -> {
                    cloud(g2, 36, 14, 0.62, CLOUD_GREY);
                    cloud(g2, 6, 30, 0.9, CLOUD_LIGHT);
                }
                case FOG -> {
                    cloud(g2, 14, 10, 0.72, CLOUD_LIGHT);
                    g2.setColor(new Color(225, 234, 244));
                    g2.setStroke(new BasicStroke(5f, BasicStroke.CAP_ROUND,
                            BasicStroke.JOIN_ROUND));
                    g2.draw(new Line2D.Double(14, 66, 84, 66));
                    g2.draw(new Line2D.Double(26, 78, 92, 78));
                    g2.draw(new Line2D.Double(14, 90, 74, 90));
                }
                case DRIZZLE -> {
                    cloud(g2, 8, 8, 0.84, CLOUD_GREY);
                    drops(g2, 3, 5f, 0.6);
                }
                case RAIN -> {
                    cloud(g2, 8, 8, 0.84, CLOUD_GREY);
                    drops(g2, 4, 6f, 1.0);
                }
                case SNOW -> {
                    cloud(g2, 8, 8, 0.84, CLOUD_GREY);
                    snowflake(g2, 30, 78);
                    snowflake(g2, 50, 88);
                    snowflake(g2, 70, 78);
                }
                case THUNDERSTORM -> {
                    cloud(g2, 8, 6, 0.84, CLOUD_DARK);
                    lightningBolt(g2);
                }
            }
        } finally {
            g2.dispose();
        }
    }

    /** Renders the icon into an image (used for the window's title-bar icon). */
    public BufferedImage toImage() {
        BufferedImage image = new BufferedImage(size, size, BufferedImage.TYPE_INT_ARGB);
        Graphics2D g2 = image.createGraphics();
        try {
            paintIcon(null, g2, 0, 0);
        } finally {
            g2.dispose();
        }
        return image;
    }

    private void sunOrMoon(Graphics2D g2, double cx, double cy, double radius) {
        if (night) {
            moon(g2, cx, cy, radius);
        } else {
            sun(g2, cx, cy, radius);
        }
    }

    private static void sun(Graphics2D g2, double cx, double cy, double radius) {
        g2.setColor(SUN);
        g2.setStroke(new BasicStroke((float) (radius * 0.22), BasicStroke.CAP_ROUND,
                BasicStroke.JOIN_ROUND));
        for (int i = 0; i < 8; i++) {
            double angle = Math.PI / 4 * i;
            g2.draw(new Line2D.Double(
                    cx + Math.cos(angle) * radius * 1.38, cy + Math.sin(angle) * radius * 1.38,
```

```
                    cx + Math.cos(angle) * radius * 1.75, cy + Math.sin(angle) * radius * 1.75));
        }
        g2.setPaint(new GradientPaint(0, (float) (cy - radius), SUN_LIGHT,
                0, (float) (cy + radius), SUN));
        g2.fill(new Ellipse2D.Double(cx - radius, cy - radius, radius * 2, radius * 2));
    }

    /** A crescent: a disc with a second, slightly smaller disc cut out of it. */
    private static void moon(Graphics2D g2, double cx, double cy, double radius) {
        Area crescent = new Area(new Ellipse2D.Double(
                cx - radius, cy - radius, radius * 2, radius * 2));
        double cutRadius = radius * 0.82;
        crescent.subtract(new Area(new Ellipse2D.Double(cx + radius * 0.45 - cutRadius,
                cy - radius * 0.3 - cutRadius, cutRadius * 2, cutRadius * 2)));
        g2.setColor(MOON);
        g2.fill(crescent);
    }

    /** A cloud made of a rounded base and two bumps; (x, y) is its top-left corner. */
    private static void cloud(Graphics2D g2, double x, double y, double scale, Color[] colors) {
        Area shape = new Area(new RoundRectangle2D.Double(0, 26, 100, 38, 38, 38));
        shape.add(new Area(new Ellipse2D.Double(14, 12, 40, 40)));
        shape.add(new Area(new Ellipse2D.Double(36, 0, 52, 52)));
        AffineTransform place = new AffineTransform();
        place.translate(x, y);
        place.scale(scale, scale);
        shape.transform(place);
        g2.setPaint(new GradientPaint(0, (float) y, colors[0],
                0, (float) (y + CLOUD_HEIGHT * scale), colors[1]));
        g2.fill(shape);
    }

    /** Slanted raindrops under a cloud. */
    private static void drops(Graphics2D g2, int count, float width, double length) {
        g2.setColor(DROP);
        g2.setStroke(new BasicStroke(width, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));
        double spacing = 56.0 / (count - 1);
        for (int i = 0; i < count; i++) {
            double x = 28 + i * spacing;
            g2.draw(new Line2D.Double(x, 70, x - 5 * length, 70 + 20 * length));
        }
    }

    /** A six-armed snowflake centred on (cx, cy). */
    private static void snowflake(Graphics2D g2, double cx, double cy) {
        g2.setColor(SNOWFLAKE);
        g2.setStroke(new BasicStroke(3f, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));
        for (int i = 0; i < 3; i++) {
            double angle = Math.PI / 3 * i;
            double dx = Math.cos(angle) * 7;
            double dy = Math.sin(angle) * 7;
            g2.draw(new Line2D.Double(cx - dx, cy - dy, cx + dx, cy + dy));
        }
    }

    private static void lightningBolt(Graphics2D g2) {
        Path2D bolt = new Path2D.Double();
        bolt.moveTo(56, 52);
        bolt.lineTo(38, 78);
        bolt.lineTo(50, 78);
        bolt.lineTo(44, 98);
        bolt.lineTo(68, 68);
        bolt.lineTo(55, 68);
        bolt.lineTo(63, 52);
        bolt.closePath();
        g2.setColor(BOLT);
        g2.fill(bolt);
    }
}
```

TimeOfDay.java

```
package weatherapp;
```

```
import java.time.Duration;
import java.time.LocalDateTime;
import java.time.LocalTime;

/**
 * The part of the day, which decides the background of the window.
 *
 * <p>When the sunrise and sunset of the searched place are known, the phase follows the sun:
 * the sky already glows before the sun rises and stays coloured for a while after it sets.
 * Without them (before the first search) the computer's clock is used.
 */
public enum TimeOfDay {
    SUNRISE, DAY, SUNSET, NIGHT;

    private static final Duration BEFORE_SUNRISE = Duration.ofMinutes(45);
    private static final Duration AFTER_SUNRISE = Duration.ofMinutes(60);
    private static final Duration BEFORE_SUNSET = Duration.ofMinutes(75);
    private static final Duration AFTER_SUNSET = Duration.ofMinutes(45);

    /**
     * The phase at a moment, given that day's sunrise and sunset in the same time zone.
     * If either is unknown (or they make no sense) the clock time is used instead.
     */
    public static TimeOfDay at(LocalDateTime now, LocalDateTime sunrise, LocalDateTime sunset) {
        if (sunrise == null || sunset == null || !sunrise.isBefore(sunset)) {
            return fromClock(now.toLocalTime());
        }
        if (within(now, sunrise.minus(BEFORE_SUNRISE), sunrise.plus(AFTER_SUNRISE))) {
            return SUNRISE;
        }
        if (within(now, sunset.minus(BEFORE_SUNSET), sunset.plus(AFTER_SUNSET))) {
            return SUNSET;
        }
        return now.isAfter(sunrise) && now.isBefore(sunset) ? DAY : NIGHT;
    }

    /** A rough phase from the hour alone: 05:00-07:00 sunrise, 17:00-19:30 sunset. */
    public static TimeOfDay fromClock(LocalTime time) {
        int minutes = time.getHour() * 60 + time.getMinute();
        if (minutes >= 5 * 60 && minutes < 7 * 60) {
            return SUNRISE;
        }
        if (minutes >= 7 * 60 && minutes < 17 * 60) {
            return DAY;
        }
        if (minutes >= 17 * 60 && minutes < 19 * 60 + 30) {
            return SUNSET;
        }
        return NIGHT;
    }

    /** True if start is not after the moment and the moment is before end. */
    private static boolean within(LocalDateTime moment, LocalDateTime start, LocalDateTime end) {
        return !moment.isBefore(start) && moment.isBefore(end);
    }
}
```

BackgroundPanel.java

```
package weatherapp;

import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.LinearGradientPaint;
import java.awt.RadialGradientPaint;
import java.awt.geom.Area;
import java.awt.geom.Ellipse2D;
import java.util.Random;

import javax.swing.JPanel;

/**
 * The window's background: a sky whose colours follow the {@link TimeOfDay}, with the sun
```

```
 * low in the sky at sunrise and sunset, a high sun by day, and a moon and stars at night.
 */
class BackgroundPanel extends JPanel {

    private static final int STAR_COUNT = 90;

    private static final float[] TWO_STOPS = {0f, 1f};
    private static final float[] THREE_STOPS = {0f, 0.55f, 1f};

    /** Star positions as fractions of the window, fixed so the sky does not flicker. */
    private final double[][] stars = new double[STAR_COUNT][3];

    private TimeOfDay timeOfDay = TimeOfDay.DAY;

    BackgroundPanel() {
        Random random = new Random(7);
        for (double[] star : stars) {
            star[0] = random.nextDouble();            // x
            star[1] = random.nextDouble() * 0.75;     // y (stars stay in the upper sky)
            star[2] = 0.8 + random.nextDouble() * 1.6; // radius in pixels
        }
    }

    TimeOfDay timeOfDay() {
        return timeOfDay;
    }

    void setTimeOfDay(TimeOfDay timeOfDay) {
        if (this.timeOfDay != timeOfDay) {
            this.timeOfDay = timeOfDay;
            repaint();
        }
    }

    @Override
    protected void paintComponent(Graphics g) {
        Graphics2D g2 = Ui.smooth((Graphics2D) g.create());
        try {
            int w = getWidth();
            int h = getHeight();
            g2.setPaint(sky(h));
            g2.fillRect(0, 0, w, h);

            switch (timeOfDay) {
                case DAY -> glowingSun(g2, w - 80, 44, 24, new Color(255, 244, 170));
                case SUNRISE -> {
                    horizonGlow(g2, w, h, new Color(255, 190, 120));
                    glowingSun(g2, w - 80, 44, 24, new Color(255, 214, 140));
                }
                case SUNSET -> {
                    horizonGlow(g2, w, h, new Color(255, 120, 70));
                    glowingSun(g2, w - 80, 44, 24, new Color(255, 170, 90));
                }
                case NIGHT -> {
                    stars(g2, w, h);
                    moon(g2, w - 80, 44, 22);
                }
            }
        } finally {
            g2.dispose();
        }
    }

    private LinearGradientPaint sky(int height) {
        return switch (timeOfDay) {
            case DAY -> new LinearGradientPaint(0, 0, 0, height, TWO_STOPS,
                    new Color[] {new Color(28, 110, 200), new Color(140, 200, 242)});
            case SUNRISE -> new LinearGradientPaint(0, 0, 0, height, THREE_STOPS,
                    new Color[] {new Color(58, 74, 150), new Color(226, 140, 154),
                        new Color(255, 205, 130)});
            case SUNSET -> new LinearGradientPaint(0, 0, 0, height, THREE_STOPS,
                    new Color[] {new Color(40, 38, 98), new Color(200, 74, 108),
                        new Color(255, 158, 74)});
            case NIGHT -> new LinearGradientPaint(0, 0, 0, height, TWO_STOPS,
                    new Color[] {new Color(6, 10, 30), new Color(28, 56, 104)});
```

```
        };
    }

    /** A sun disc surrounded by a soft glow. */
    private static void glowingSun(Graphics2D g2, double cx, double cy, double radius,
                                   Color color) {
        double glow = radius * 3.2;
        g2.setPaint(new RadialGradientPaint((float) cx, (float) cy, (float) glow, TWO_STOPS,
                new Color[] {Ui.withAlpha(color, 170), Ui.withAlpha(color, 0)}));
        g2.fill(new Ellipse2D.Double(cx - glow, cy - glow, glow * 2, glow * 2));
        g2.setColor(Ui.withAlpha(color, 245));
        g2.fill(new Ellipse2D.Double(cx - radius, cy - radius, radius * 2, radius * 2));
    }

    /** A warm glow along the bottom edge, where the sun is rising or setting. */
    private static void horizonGlow(Graphics2D g2, int w, int h, Color color) {
        float radius = w * 0.75f;
        g2.setPaint(new RadialGradientPaint(w / 2f, h * 1.05f, radius, TWO_STOPS,
                new Color[] {Ui.withAlpha(color, 190), Ui.withAlpha(color, 0)}));
        g2.fillRect(0, 0, w, h);
    }

    private void stars(Graphics2D g2, int w, int h) {
        g2.setColor(new Color(255, 255, 255, 190));
        for (double[] star : stars) {
            double r = star[2];
            g2.fill(new Ellipse2D.Double(star[0] * w - r, star[1] * h - r, r * 2, r * 2));
        }
    }

    /** A crescent moon with a faint halo. */
    private static void moon(Graphics2D g2, double cx, double cy, double radius) {
        double halo = radius * 3;
        g2.setPaint(new RadialGradientPaint((float) cx, (float) cy, (float) halo, TWO_STOPS,
                new Color[] {new Color(200, 215, 255, 90), new Color(200, 215, 255, 0)}));
        g2.fill(new Ellipse2D.Double(cx - halo, cy - halo, halo * 2, halo * 2));

        Area crescent = new Area(new Ellipse2D.Double(
                cx - radius, cy - radius, radius * 2, radius * 2));
        double cut = radius * 0.85;
        crescent.subtract(new Area(new Ellipse2D.Double(
                cx + radius * 0.45 - cut, cy - radius * 0.25 - cut, cut * 2, cut * 2)));
        g2.setColor(new Color(250, 240, 190));
        g2.fill(crescent);
    }
}
```

SearchHistory.java

```
package weatherapp;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.LocalDateTime;
import java.time.format.DateTimeParseException;
import java.util.ArrayList;
import java.util.List;

/**
 * The list of recent searches, newest first, each with the time it was made.
 *
 * <p>The list is kept in a small tab-separated text file so that it survives restarting the
 * app. Saving is a convenience: if the file cannot be written the history still works for
 * the current session, and unreadable lines in an existing file are skipped.
 */
public final class SearchHistory {

    /** One past search. */
    public record Entry(LocalDateTime time, Location location) { }

    private final Path file;
    private final int capacity;
```

```
    private final List<Entry> entries = new ArrayList<>();

    /**
     * @param file     where the history is stored (created on first save)
     * @param capacity how many searches to keep; older ones are dropped
     */
    public SearchHistory(Path file, int capacity) {
        this.file = file;
        this.capacity = capacity;
        load();
    }

    /** The entries, newest first. */
    public List<Entry> entries() {
        return List.copyOf(entries);
    }

    /** Adds a search at the top of the list. */
    public void record(Location location, LocalDateTime when) {
        entries.add(0, new Entry(when, location));
        while (entries.size() > capacity) {
            entries.remove(entries.size() - 1);
        }
        save();
    }

    public void clear() {
        entries.clear();
        save();
    }

    private void load() {
        if (!Files.isRegularFile(file)) {
            return;
        }
        try {
            for (String line : Files.readAllLines(file, StandardCharsets.UTF_8)) {
                Entry entry = parseLine(line);
                if (entry != null && entries.size() < capacity) {
                    entries.add(entry);
                }
            }
        } catch (IOException e) {
            System.err.println("Could not read the search history: " + e.getMessage());
        }
    }

    private void save() {
        List<String> lines = new ArrayList<>();
        for (Entry entry : entries) {
            Location place = entry.location();
            lines.add(String.join("\t", entry.time().toString(), clean(place.name()),
                    clean(place.region()), clean(place.country()),
                    Double.toString(place.latitude()), Double.toString(place.longitude())));
        }
        try {
            Files.createDirectories(file.toAbsolutePath().getParent());
            Files.write(file, lines, StandardCharsets.UTF_8);
        } catch (IOException e) {
            System.err.println("Could not save the search history: " + e.getMessage());
        }
    }

    /** Returns null for a line that is not a valid history entry. */
    private static Entry parseLine(String line) {
        String[] parts = line.split("\t", -1);
        if (parts.length != 6) {
            return null;
        }
        try {
            Location place = new Location(parts[1], parts[2], parts[3],
                    Double.parseDouble(parts[4]), Double.parseDouble(parts[5]));
            return new Entry(LocalDateTime.parse(parts[0]), place);
        } catch (NumberFormatException | DateTimeParseException e) {
            return null;
```

```
        }
    }

    /** Tabs and line breaks would corrupt the file format. */
    private static String clean(String text) {
        return text.replaceAll("[\\t\\r\\n]+", " ");
    }
}
```

CurrentWeatherPanel.java

```
package weatherapp;

import java.awt.BorderLayout;
import java.awt.CardLayout;
import java.awt.Color;
import java.awt.Font;
import java.awt.GridLayout;

import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JLabel;
import javax.swing.JPanel;

/**
 * The large card with the weather right now: place, local time, icon, temperature,
 * conditions and a grid of details. Before the first search it shows a short welcome note.
 */
class CurrentWeatherPanel extends RoundedPanel {

    private static final String WELCOME = "welcome";
    private static final String WEATHER = "weather";

    private final CardLayout cards = new CardLayout();
    private final JPanel content = new JPanel(cards);

    private final JLabel place = Ui.label(" ", Font.BOLD, 24, Ui.TEXT);
    private final JLabel localTime = Ui.label(" ", Font.PLAIN, 14, Ui.TEXT_MUTED);
    private final JLabel icon = new JLabel();
    private final JLabel temperature = Ui.label(" ", Font.BOLD, 48, Ui.TEXT);
    private final JLabel conditions = Ui.label(" ", Font.PLAIN, 18, Ui.TEXT);

    private final DetailTile feelsLike = new DetailTile("Feels like");
    private final DetailTile humidity = new DetailTile("Humidity");
    private final DetailTile wind = new DetailTile("Wind");
    private final DetailTile pressure = new DetailTile("Pressure");
    private final DetailTile sunrise = new DetailTile("Sunrise");
    private final DetailTile sunset = new DetailTile("Sunset");

    CurrentWeatherPanel() {
        super(new BorderLayout(), Ui.CARD);
        setBorder(BorderFactory.createEmptyBorder(14, 20, 16, 20));
        content.setOpaque(false);
        content.add(buildWelcome(), WELCOME);
        content.add(buildWeather(), WEATHER);
        add(content, BorderLayout.CENTER);
    }

    private JPanel buildWelcome() {
        JPanel welcome = new JPanel();
        welcome.setOpaque(false);
        welcome.setLayout(new BoxLayout(welcome, BoxLayout.Y_AXIS));

        JLabel picture = new JLabel(new WeatherIcon(IconType.PARTLY_CLOUDY, false, 96));
        picture.setAlignmentX(CENTER_ALIGNMENT);

        JLabel text = new JLabel("<html><div style='text-align:center'>"
                + "<span style='font-size:20px'><b>Welcome to the Weather Information App"
                + "</b></span><br><br>Type a city (for example <b>Paris, France</b>) or "
                + "coordinates (for example <b>48.85, 2.35</b>)<br>and press <b>Search</b> to "
                + "see the current weather and the 5-day forecast.</div></html>");
        text.setFont(Ui.font(Font.PLAIN, 15));
        text.setForeground(Ui.TEXT);
```

```
        text.setAlignmentX(CENTER_ALIGNMENT);

        welcome.add(Box.createVerticalGlue());
        welcome.add(picture);
        welcome.add(Box.createVerticalStrut(12));
        welcome.add(text);
        welcome.add(Box.createVerticalGlue());
        return welcome;
    }

    private JPanel buildWeather() {
        JPanel header = new JPanel();
        header.setOpaque(false);
        header.setLayout(new BoxLayout(header, BoxLayout.Y_AXIS));
        header.add(place);
        header.add(localTime);

        // Icon and temperature on the left, the six detail tiles on the right.
        JPanel readings = new JPanel(new BorderLayout(14, 0));
        readings.setOpaque(false);
        JPanel temperatureColumn = new JPanel();
        temperatureColumn.setOpaque(false);
        temperatureColumn.setLayout(new BoxLayout(temperatureColumn, BoxLayout.Y_AXIS));
        temperatureColumn.add(Box.createVerticalGlue());
        temperatureColumn.add(temperature);
        temperatureColumn.add(conditions);
        temperatureColumn.add(Box.createVerticalGlue());
        readings.add(icon, BorderLayout.WEST);
        readings.add(temperatureColumn, BorderLayout.CENTER);

        JPanel details = new JPanel(new GridLayout(2, 3, 10, 10));
        details.setOpaque(false);
        for (DetailTile tile : new DetailTile[] {feelsLike, humidity, wind,
                pressure, sunrise, sunset}) {
            details.add(tile);
        }

        JPanel body = new JPanel(new BorderLayout(20, 0));
        body.setOpaque(false);
        body.add(readings, BorderLayout.WEST);
        body.add(details, BorderLayout.CENTER);

        JPanel weather = new JPanel(new BorderLayout(0, 10));
        weather.setOpaque(false);
        weather.add(header, BorderLayout.NORTH);
        weather.add(body, BorderLayout.CENTER);
        return weather;
    }

    /** Shows a report in the chosen units. */
    void show(WeatherReport report, TemperatureUnit temperatureUnit, WindSpeedUnit windUnit) {
        CurrentConditions now = report.current();
        DailyForecast today = report.today();

        place.setText(report.location().label());
        localTime.setText("Local time: " + DisplayFormat.dateTime(now.time())
                + (report.timezone().isEmpty() ? "" : "  (" + report.timezone() + ")"));
        icon.setIcon(new WeatherIcon(WeatherCode.iconFor(now.weatherCode()),
                !now.daytime(), 96));
        temperature.setText(DisplayFormat.temperature(now.temperatureC(), temperatureUnit, 1));
        conditions.setText(WeatherCode.describe(now.weatherCode()));

        feelsLike.setValue(DisplayFormat.temperature(now.feelsLikeC(), temperatureUnit, 1));
        humidity.setValue(DisplayFormat.percent(now.humidityPercent()));
        wind.setCaption("Wind (" + DisplayFormat.compassPoint(now.windDirectionDegrees()) + ")");
        wind.setValue(DisplayFormat.windSpeed(now.windSpeedKmh(), windUnit));
        pressure.setValue(DisplayFormat.pressure(now.pressureHpa()));
        sunrise.setValue(DisplayFormat.clockTime(today.sunrise()));
        sunset.setValue(DisplayFormat.clockTime(today.sunset()));
        cards.show(content, WEATHER);
    }

    /** One labelled value in the details grid. */
    private static final class DetailTile extends RoundedPanel {
```

```
        private final JLabel caption;
        private final JLabel value = Ui.label(" ", Font.BOLD, 17, Ui.TEXT);

        DetailTile(String captionText) {
            super(new BorderLayout(), new Color(255, 255, 255, 28));
            caption = Ui.label(captionText, Font.PLAIN, 13, Ui.TEXT_MUTED);
            setBorder(BorderFactory.createEmptyBorder(8, 12, 8, 12));
            add(caption, BorderLayout.NORTH);
            add(value, BorderLayout.CENTER);
        }

        void setCaption(String text) {
            caption.setText(text);
        }

        void setValue(String text) {
            value.setText(text);
        }
    }
}
```

ForecastPanel.java

```
package weatherapp;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Font;
import java.awt.GridLayout;
import java.time.LocalDate;
import java.util.List;

import javax.swing.BorderFactory;
import javax.swing.BoxLayout;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.SwingConstants;

/** The row of day cards that shows the short-term forecast. */
class ForecastPanel extends RoundedPanel {

    private final JPanel row = new JPanel(new GridLayout(1, WeatherService.FORECAST_DAYS, 10, 0));

    ForecastPanel() {
        super(new BorderLayout(0, 8), Ui.CARD);
        setBorder(BorderFactory.createEmptyBorder(12, 16, 16, 16));
        row.setOpaque(false);
        add(Ui.label("5-day forecast", Font.BOLD, 16, Ui.TEXT), BorderLayout.NORTH);
        add(row, BorderLayout.CENTER);
        row.add(placeholder());
    }

    private static JLabel placeholder() {
        JLabel label = Ui.label("The forecast appears here after your first search.",
                Font.PLAIN, 14, Ui.TEXT_MUTED);
        label.setHorizontalAlignment(SwingConstants.CENTER);
        return label;
    }

    /** Replaces the cards with the given forecast, in the chosen temperature unit. */
    void show(List<DailyForecast> forecast, TemperatureUnit unit) {
        row.removeAll();
        row.setLayout(new GridLayout(1, Math.max(1, forecast.size()), 10, 0));
        LocalDate today = forecast.get(0).date();
        for (DailyForecast day : forecast) {
            row.add(new DayCard(day, today, unit));
        }
        row.revalidate();
        row.repaint();
    }

    /** One day: weekday, date, icon, high and low, and the chance of precipitation. */
    private static final class DayCard extends RoundedPanel {
```

```
        DayCard(DailyForecast day, LocalDate today, TemperatureUnit unit) {
            super(new BorderLayout(), new Color(255, 255, 255, 30));
            setBorder(BorderFactory.createEmptyBorder(10, 6, 10, 6));

            JPanel heading = new JPanel();
            heading.setOpaque(false);
            heading.setLayout(new BoxLayout(heading, BoxLayout.Y_AXIS));
            heading.add(centered(DisplayFormat.dayName(day.date(), today), Font.BOLD, 16,
                    Ui.TEXT));
            heading.add(centered(DisplayFormat.dayAndMonth(day.date()), Font.PLAIN, 12,
                    Ui.TEXT_MUTED));

            JLabel picture = new JLabel(
                    new WeatherIcon(WeatherCode.iconFor(day.weatherCode()), false, 50));
            picture.setHorizontalAlignment(SwingConstants.CENTER);
            picture.setToolTipText(WeatherCode.describe(day.weatherCode()));

            JPanel numbers = new JPanel();
            numbers.setOpaque(false);
            numbers.setLayout(new BoxLayout(numbers, BoxLayout.Y_AXIS));
            numbers.add(centered(DisplayFormat.temperature(day.highC(), unit, 0) + " / "
                    + DisplayFormat.temperature(day.lowC(), unit, 0), Font.BOLD, 14, Ui.TEXT));
            numbers.add(centered(WeatherCode.describe(day.weatherCode()), Font.PLAIN, 12,
                    Ui.TEXT_MUTED));
            numbers.add(centered(day.precipitationChancePercent() == DailyForecast.UNKNOWN_CHANCE
                    ? " " : "Precipitation " + DisplayFormat.percent(
                            day.precipitationChancePercent()), Font.PLAIN, 12, Ui.TEXT_MUTED));

            add(heading, BorderLayout.NORTH);
            add(picture, BorderLayout.CENTER);
            add(numbers, BorderLayout.SOUTH);
        }

        private static JLabel centered(String text, int style, int size, Color color) {
            JLabel label = Ui.label(text, style, size, color);
            label.setAlignmentX(CENTER_ALIGNMENT);
            label.setHorizontalAlignment(SwingConstants.CENTER);
            return label;
        }
    }
}
```

HistoryPanel.java

```
package weatherapp;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Component;
import java.awt.Cursor;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.util.List;
import java.util.function.Consumer;

import javax.swing.BorderFactory;
import javax.swing.DefaultListModel;
import javax.swing.JLabel;
import javax.swing.JList;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.ListCellRenderer;
import javax.swing.ListSelectionModel;

/** The side card listing recent searches with their timestamps; click one to show it again. */
class HistoryPanel extends RoundedPanel {

    private static final int WIDTH = 270;
    private static final int CELL_WIDTH = 220;
```

```
    private final DefaultListModel<SearchHistory.Entry> model = new DefaultListModel<>();
    private final JList<SearchHistory.Entry> list = new JList<>(model);
    private final JLabel emptyNote = Ui.label("No searches yet.", Font.PLAIN, 14, Ui.TEXT_MUTED);

    /**
     * @param onSelect called with the entry the user clicks (or presses Enter on)
     * @param onClear  called when the user presses "Clear history"
     */
    HistoryPanel(Consumer<SearchHistory.Entry> onSelect, Runnable onClear) {
        super(new BorderLayout(0, 8), Ui.CARD);
        setBorder(BorderFactory.createEmptyBorder(12, 14, 14, 14));
        setPreferredSize(new Dimension(WIDTH, 0));

        list.setOpaque(false);
        list.setFixedCellWidth(CELL_WIDTH);        // long names are cut with "..."
        list.setCellRenderer(new EntryRenderer());
        list.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        list.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        list.setToolTipText("Click a search to look it up again");
        list.addMouseListener(new MouseAdapter() {
            @Override
            public void mouseClicked(MouseEvent e) {
                int index = list.locationToIndex(e.getPoint());
                if (index >= 0 && list.getCellBounds(index, index).contains(e.getPoint())) {
                    onSelect.accept(model.get(index));
                }
            }
        });
        list.addKeyListener(new KeyAdapter() {
            @Override
            public void keyPressed(KeyEvent e) {
                if (e.getKeyCode() == KeyEvent.VK_ENTER && list.getSelectedValue() != null) {
                    onSelect.accept(list.getSelectedValue());
                }
            }
        });

        JScrollPane scroll = new JScrollPane(list);
        scroll.setOpaque(false);
        scroll.getViewport().setOpaque(false);
        scroll.setBorder(BorderFactory.createEmptyBorder());
        scroll.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_NEVER);
        scroll.getVerticalScrollBar().setUI(new ThinScrollBarUI());
        scroll.getVerticalScrollBar().setOpaque(false);

        AccentButton clear = new AccentButton("Clear history");
        clear.addActionListener(e -> onClear.run());

        JPanel top = new JPanel(new BorderLayout());
        top.setOpaque(false);
        top.add(Ui.label("Recent searches", Font.BOLD, 16, Ui.TEXT), BorderLayout.NORTH);
        top.add(emptyNote, BorderLayout.SOUTH);

        add(top, BorderLayout.NORTH);
        add(scroll, BorderLayout.CENTER);
        add(clear, BorderLayout.SOUTH);
    }

    /** Replaces the list with the given entries (newest first). */
    void refresh(List<SearchHistory.Entry> entries) {
        model.clear();
        entries.forEach(model::addElement);
        emptyNote.setVisible(entries.isEmpty());
    }

    int entryCount() {
        return model.size();
    }

    /** Draws one entry as a place name above a smaller timestamp. */
    private static final class EntryRenderer extends JPanel
            implements ListCellRenderer<SearchHistory.Entry> {

        private final JLabel place = Ui.label(" ", Font.BOLD, 14, Ui.TEXT);
        private final JLabel time = Ui.label(" ", Font.PLAIN, 12, Ui.TEXT_MUTED);
```

```
        EntryRenderer() {
            super(new BorderLayout());
            setBorder(BorderFactory.createEmptyBorder(6, 8, 6, 8));
            add(place, BorderLayout.NORTH);
            add(time, BorderLayout.SOUTH);
        }

        @Override
        public Component getListCellRendererComponent(JList<? extends SearchHistory.Entry> list,
                SearchHistory.Entry entry, int index, boolean selected, boolean focused) {
            place.setText(entry.location().label());
            time.setText(DisplayFormat.historyTime(entry.time()));
            setOpaque(selected);                        // only the selected row has a fill
            setBackground(new Color(55, 85, 140));
            return this;
        }
    }
}
```

StatusBar.java

```
package weatherapp;

import java.awt.BorderLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JLabel;

/** The message line under the search box: progress and results in blue, errors in red. */
class StatusBar extends RoundedPanel {

    private final JLabel message = Ui.label("", Font.PLAIN, 15, Ui.TEXT);

    StatusBar() {
        super(new BorderLayout(), Ui.CARD);
        setBorder(BorderFactory.createEmptyBorder(9, 16, 9, 16));
        add(message, BorderLayout.CENTER);
    }

    void showInfo(String text) {
        message.setText(text);
        setFill(Ui.CARD);
    }

    void showError(WeatherException problem) {
        message.setText(problem.type().title() + ": " + problem.getMessage());
        setFill(Ui.ERROR);
    }

    /** Reports a bug (an exception nobody expected) without exposing technical details. */
    void showUnexpectedError() {
        message.setText("Something went wrong inside the app. Please try the search again.");
        setFill(Ui.ERROR);
    }

    String text() {
        return message.getText();
    }
}
```

RoundedPanel.java

```
package weatherapp;

import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.LayoutManager;
import java.awt.geom.RoundRectangle2D;

import javax.swing.JPanel;
```

```
/** A panel with rounded corners and a translucent fill, so the background shows through. */
class RoundedPanel extends JPanel {

    private static final int ARC = 22;

    private Color fill;

    RoundedPanel(LayoutManager layout, Color fill) {
        super(layout);
        this.fill = fill;
        setOpaque(false);
    }

    /** Changes the fill colour (used by the status bar to show errors in red). */
    void setFill(Color fill) {
        this.fill = fill;
        repaint();
    }

    @Override
    protected void paintComponent(Graphics g) {
        Graphics2D g2 = Ui.smooth((Graphics2D) g.create());
        try {
            RoundRectangle2D shape = new RoundRectangle2D.Double(
                    0.5, 0.5, getWidth() - 1.0, getHeight() - 1.0, ARC, ARC);
            g2.setColor(fill);
            g2.fill(shape);
            g2.setColor(Ui.CARD_BORDER);
            g2.draw(shape);
        } finally {
            g2.dispose();
        }
        super.paintComponent(g);
    }
}
```

HintTextField.java

```
package weatherapp;

import java.awt.Color;
import java.awt.Font;
import java.awt.FontMetrics;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.Insets;

import javax.swing.BorderFactory;
import javax.swing.JTextField;

/** A text field that shows a grey hint while it is empty. */
class HintTextField extends JTextField {

    private static final Color HINT_COLOR = new Color(120, 130, 145);

    private final String hint;

    HintTextField(String hint, int columns) {
        super(columns);
        this.hint = hint;
        setFont(Ui.font(Font.PLAIN, 16));
        setBorder(BorderFactory.createCompoundBorder(
                BorderFactory.createLineBorder(new Color(255, 255, 255, 120)),
                BorderFactory.createEmptyBorder(8, 12, 8, 12)));
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        if (getText().isEmpty()) {
            Graphics2D g2 = Ui.smooth((Graphics2D) g.create());
            try {
                Insets insets = getInsets();
                FontMetrics metrics = g2.getFontMetrics(getFont());
```

```
                g2.setColor(HINT_COLOR);
                g2.setFont(getFont());
                g2.drawString(hint, insets.left,
                        (getHeight() - metrics.getHeight()) / 2 + metrics.getAscent());
            } finally {
                g2.dispose();
            }
        }
    }
}
```

AccentButton.java

```
package weatherapp;

import java.awt.Color;
import java.awt.Cursor;
import java.awt.Font;
import java.awt.FontMetrics;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.geom.RoundRectangle2D;

import javax.swing.BorderFactory;
import javax.swing.JButton;

/** A flat, rounded button in the app's accent colour. */
class AccentButton extends JButton {

    private static final Color DISABLED = new Color(110, 120, 140);

    AccentButton(String text) {
        super(text);
        setFont(Ui.font(Font.BOLD, 15));
        setForeground(Color.WHITE);
        setContentAreaFilled(false);
        setBorderPainted(false);
        setFocusPainted(false);
        setBorder(BorderFactory.createEmptyBorder(9, 26, 9, 26));
        setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    }

    @Override
    protected void paintComponent(Graphics g) {
        Graphics2D g2 = Ui.smooth((Graphics2D) g.create());
        try {
            Color fill = Ui.ACCENT;
            if (!isEnabled()) {
                fill = DISABLED;
            } else if (getModel().isPressed()) {
                fill = Ui.ACCENT.darker();
            } else if (getModel().isRollover()) {
                fill = Ui.ACCENT.brighter();
            }
            g2.setColor(fill);
            g2.fill(new RoundRectangle2D.Double(0, 0, getWidth(), getHeight(), 14, 14));

            g2.setFont(getFont());
            g2.setColor(Color.WHITE);
            FontMetrics metrics = g2.getFontMetrics();
            g2.drawString(getText(), (getWidth() - metrics.stringWidth(getText())) / 2,
                    (getHeight() - metrics.getHeight()) / 2 + metrics.getAscent());
        } finally {
            g2.dispose();
        }
    }
}
```

SegmentedControl.java

```
package weatherapp;

import java.awt.Color;
import java.awt.Cursor;
```

```
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.FontMetrics;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.geom.RoundRectangle2D;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Consumer;
import java.util.function.Function;

import javax.swing.BorderFactory;
import javax.swing.ButtonGroup;
import javax.swing.JPanel;
import javax.swing.JToggleButton;

/**
 * A row of mutually exclusive options drawn as one pill, for choosing a unit.
 *
 * @param <T> the type of the options (here an enum of units)
 */
class SegmentedControl<T> extends JPanel {

    private final Map<T, JToggleButton> buttons = new HashMap<>();
    private final Consumer<T> onChange;

    /**
     * @param options  the choices, in display order; the first is selected at the start
     * @param caption  turns an option into its label
     * @param onChange called whenever the selection changes
     */
    SegmentedControl(List<T> options, Function<T, String> caption, Consumer<T> onChange) {
        super(new FlowLayout(FlowLayout.LEFT, 0, 0));
        this.onChange = onChange;
        setOpaque(false);

        ButtonGroup group = new ButtonGroup();
        for (T option : options) {
            JToggleButton button = new Segment(caption.apply(option));
            button.addActionListener(e -> onChange.accept(option));
            group.add(button);
            buttons.put(option, button);
            add(button);
        }
        buttons.get(options.get(0)).setSelected(true);
    }

    /** Selects an option as if the user had clicked it (the change listener is called). */
    void select(T option) {
        JToggleButton button = buttons.get(option);
        if (!button.isSelected()) {
            button.setSelected(true);
            onChange.accept(option);
        }
    }

    @Override
    protected void paintComponent(Graphics g) {
        Graphics2D g2 = Ui.smooth((Graphics2D) g.create());
        try {
            g2.setColor(new Color(255, 255, 255, 38));
            g2.fill(new RoundRectangle2D.Double(0, 0, getWidth(), getHeight(), 16, 16));
        } finally {
            g2.dispose();
        }
        super.paintComponent(g);
    }

    /** One choice of the control: highlighted when selected. */
    private static final class Segment extends JToggleButton {

        Segment(String text) {
            super(text);
            setFont(Ui.font(Font.BOLD, 14));
```

```
            setContentAreaFilled(false);
            setBorderPainted(false);
            setFocusPainted(false);
            setBorder(BorderFactory.createEmptyBorder(6, 14, 6, 14));
            setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        }

        @Override
        protected void paintComponent(Graphics g) {
            Graphics2D g2 = Ui.smooth((Graphics2D) g.create());
            try {
                if (isSelected()) {
                    g2.setColor(Ui.ACCENT);
                    g2.fill(new RoundRectangle2D.Double(
                            2, 2, getWidth() - 4, getHeight() - 4, 14, 14));
                }
                g2.setFont(getFont());
                g2.setColor(isSelected() ? Color.WHITE : Ui.TEXT_MUTED);
                FontMetrics metrics = g2.getFontMetrics();
                g2.drawString(getText(), (getWidth() - metrics.stringWidth(getText())) / 2,
                        (getHeight() - metrics.getHeight()) / 2 + metrics.getAscent());
            } finally {
                g2.dispose();
            }
        }
    }
}
```

ThinScrollBarUI.java

```
package weatherapp;

import java.awt.Color;
import java.awt.Dimension;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.Rectangle;
import java.awt.geom.RoundRectangle2D;

import javax.swing.JButton;
import javax.swing.JComponent;
import javax.swing.plaf.basic.BasicScrollBarUI;

/** A slim, translucent scroll bar that suits the window's glass look. */
final class ThinScrollBarUI extends BasicScrollBarUI {

    private static final int WIDTH = 8;

    @Override
    protected void configureScrollBarColors() {
        super.configureScrollBarColors();
        thumbColor = new Color(255, 255, 255, 120);
    }

    @Override
    public Dimension getPreferredSize(JComponent c) {
        return new Dimension(WIDTH + 4, super.getPreferredSize(c).height);
    }

    @Override
    protected JButton createDecreaseButton(int orientation) {
        return invisibleButton();
    }

    @Override
    protected JButton createIncreaseButton(int orientation) {
        return invisibleButton();
    }

    @Override
    protected void paintTrack(Graphics g, JComponent c, Rectangle bounds) {
        // The track stays transparent.
    }
```

```
    @Override
    protected void paintThumb(Graphics g, JComponent c, Rectangle bounds) {
        Graphics2D g2 = Ui.smooth((Graphics2D) g.create());
        try {
            g2.setColor(thumbColor);
            g2.fill(new RoundRectangle2D.Double(
                    bounds.x + 2, bounds.y, WIDTH, bounds.height, WIDTH, WIDTH));
        } finally {
            g2.dispose();
        }
    }

    /** The arrow buttons are not needed; wheel and dragging are enough. */
    private static JButton invisibleButton() {
        JButton button = new JButton();
        button.setPreferredSize(new Dimension(0, 0));
        return button;
    }
}
```

Ui.java

```
package weatherapp;

import java.awt.Color;
import java.awt.Font;
import java.awt.Graphics2D;
import java.awt.RenderingHints;

import javax.swing.JLabel;

/** Colours, fonts and small helpers shared by all the window's components. */
final class Ui {

    /** The translucent dark "glass" behind every group of controls. */
    static final Color CARD = new Color(8, 18, 42, 150);
    static final Color CARD_BORDER = new Color(255, 255, 255, 50);
    static final Color TEXT = Color.WHITE;
    static final Color TEXT_MUTED = new Color(203, 213, 225);
    static final Color ACCENT = new Color(37, 99, 235);
    static final Color ERROR = new Color(185, 28, 28, 230);

    private static final String FONT_FAMILY = "Segoe UI";

    private Ui() {
    }

    static Font font(int style, int size) {
        return new Font(FONT_FAMILY, style, size);
    }

    /** A transparent label with the given look. */
    static JLabel label(String text, int style, int size, Color color) {
        JLabel label = new JLabel(text);
        label.setFont(font(style, size));
        label.setForeground(color);
        return label;
    }

    /** Switches on smooth edges and text for custom painting. */
    static Graphics2D smooth(Graphics2D g) {
        g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        g.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
                RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
        g.setRenderingHint(RenderingHints.KEY_STROKE_CONTROL, RenderingHints.VALUE_STROKE_PURE);
        return g;
    }

    /** The same colour with a different opacity (0 to 255). */
    static Color withAlpha(Color color, int alpha) {
        return new Color(color.getRed(), color.getGreen(), color.getBlue(), alpha);
    }
}
```